Microsoft Research
Interview Preparation

Research Intern — Quantum Applications

Job #200023796 · Redmond, WA

Edoardo Spigarolo

M.Sc. Quantum Science & Engineering, EPFL

Open Quantum Institute @ CERN

Prepared: February 2026

Scope

Chemistry and physics applications for early fault-tolerant quantum computers.
Molecular simulation · Materials discovery · Strongly correlated systems.
Logical qubits · Surface-code operations · Resource estimation.

Contents

I Interview Format & Questions **2**

1 Interview Format **2**

2 Interview Questions **2**
2.1 Quantum Fundamentals (15 Questions) . . . 2
2.2 VQE & Variational Algorithms (15 Questions) . . . 3
2.3 Quantum Error Correction (15 Questions) . . . 3
2.4 Computational Chemistry on Quantum Computers (15 Questions) . . . 4
2.5 Microsoft-Specific Questions (15 Questions) . . . 4
2.6 Circuit-Based QC & Algorithms (10 Questions) . . . 5
2.7 Your Background & Fit (10 Questions) . . . 5

II Study Material & Crash Courses **6**

3 VQE (Variational Quantum Eigensolver) **6**
3.1 Core Concept . . . 6
3.2 Hamiltonian Encoding . . . 6
3.3 Ansatz Types . . . 6
3.4 VQE Code Example (PennyLane) . . . 7
3.5 VQE vs QPE Comparison . . . 7

4 Quantum Error Correction & Surface Codes **7**
4.1 Why QEC? . . . 7
4.2 Stabilizer Formalism . . . 7
4.3 The Surface Code . . . 8
4.4 Logical Operations . . . 8
4.5 Magic State Distillation . . . 8
4.6 Beyond Surface Codes . . . 8

5 Computational Chemistry on Quantum Computers **8**
5.1 The Pipeline . . . 8
5.2 Key Concepts . . . 9
5.3 MS+Quantinuum Chemistry Demonstration (Sep 2024) . . . 9

6 Circuit-Based QC Fundamentals **9**
6.1 Annealing vs. Gates . . . 9
6.2 Essential Gates . . . 9
6.3 Key Algorithms . . . 9

7 Microsoft Quantum Stack **10**
7.1 Roadmap . . . 10
7.2 Majorana 1 — The Topological Qubit . . . 10
7.3 Azure Quantum Platform . . . 10
7.4 Key People . . . 10

III 7-Day Study Plan **11**

IV Comprehensive Answer Key 13

8 Quantum Fundamentals — Answers 13

9 VQE & Variational Algorithms — Answers 17

10 Quantum Error Correction — Answers 22

11 Computational Chemistry — Answers 25

12 Microsoft-Specific — Answers 28

13 Circuit-Based QC & Algorithms — Answers 31

14 Background & Behavioral — Answers 32

Appendix: Complete Resource List 34

V Hands-On Coding Projects 36

A Project 1: VQE for H_2 — Full Pipeline Day 1 36

B Project 2: Potential Energy Surface & Bond Dissociation Day 1 38

C Project 3: Ansätze Comparison & Barren Plateaus Day 2 41

D Project 4: Quantum Phase Estimation from Scratch Day 2 44

E Project 5: Quantum Error Correction Simulation Day 3 47

F Project 6: Magic State Distillation Analysis Day 3 50

G Project 7: Azure Quantum Resource Estimation Day 4 53

H Project 8: VQE for LiH with Qubit Reduction Day 5 56

I Project 9: Classical Shadows for Efficient Measurement Day 5 59

J Project 10: Trotter-Suzuki Time Evolution Day 6 62

K Project 11: Noise Simulation & Zero-Noise Extrapolation Day 6 66

L Project 12: ADAPT-VQE for Efficient Ansatz Construction Day 6 70

M Project 13: Quantum Teleportation & Gate Teleportation Day 7 74

N Project 14: Grover's Search Algorithm Day 7 77

Part I

Interview Format & Questions

1 Interview Format

MSR quantum internship interviews typically involve 2–3 rounds:

Round 1: Initial Screen (30–60 min) — Phone/video call with the hiring manager. Background, motivation, high-level quantum knowledge.

Round 2: Technical Deep-Dive (1–3 sessions, 45–60 min each) — Whiteboard/coding problems, past research discussion, problem-solving exercises.

Round 3: Research Presentation (optional) — Present thesis work or a chosen research topic.

Round 4: Behavioral/Culture Fit — Growth mindset, collaboration, intellectual curiosity.

Key Details — Job #200023796

Duration:	12-week internship
Location:	Redmond, WA (3 days/week in-office)
Pay (IC2):	\$5,610–\$11,010/month
Pay (IC3):	\$6,710–\$13,270/month
Submission:	2+ reference letters + cover letter + work samples

2 Interview Questions

2.1 Quantum Fundamentals (15 Questions)

Category: Core Quantum Mechanics & Information

Q1. What is a qubit? How does it differ from a classical bit? Explain superposition.
Q2. Explain quantum entanglement. What is a Bell state?
Q3. Describe the Bloch sphere representation of a single qubit.
Q4. What is the no-cloning theorem and why is it important for quantum computing?
Q5. What are T_1 and T_2 relaxation times? How do they affect computation?
Q6. What is the difference between gate-based and adiabatic/annealing quantum computing?
Q7. How does the adiabatic theorem connect to quantum annealing? Under what conditions does it fail?
Q8. Explain the concept of quantum decoherence. What are the main decoherence channels?
Q9. What is a density matrix? When is it preferable to a state vector description?
Q10. What is the Gottesman-Knill theorem and what does it imply about quantum speedup?
Q11. Explain the difference between a pure state and a mixed state.
Q12. What is quantum teleportation? Why doesn't it violate no-cloning?
Q13. What is the quantum Zeno effect?
Q14. Explain the connection between the Ising model and QUBO formulation.
Q15. What is a tensor product and how does it relate to the Hilbert space of composite systems?

2.2 VQE & Variational Algorithms (15 Questions)

Category: Variational Quantum Eigensolver — HIGH PRIORITY

Q16. Explain the Variational Quantum Eigensolver (VQE). What is the variational principle?
Q17. What is a parameterized quantum circuit (ansatz)? Describe UCCSD.
Q18. How does the Jordan-Wigner transformation work? Map a simple example.
Q19. Compare Jordan-Wigner and Bravyi-Kitaev transformations. When would you prefer one over the other?
Q20. What is the difference between VQE and QPE for chemistry simulation?
Q21. How would you choose an active space for a molecular simulation?
Q22. What are barren plateaus? How do they affect VQE optimization?
Q23. Explain the parameter-shift rule for computing gradients on quantum hardware.
Q24. What is ADAPT-VQE and how does it improve upon standard VQE?
Q25. How does the number of Pauli terms in a molecular Hamiltonian scale with system size?
Q26. What are classical shadows and how do they reduce measurement overhead?
Q27. What is the Hartree-Fock state and why is it used as a starting point for VQE?
Q28. Explain qubit tapering. How does molecular symmetry reduce qubit count?
Q29. What is quantum subspace expansion (QSE)?
Q30. How would you benchmark a VQE result against classical methods (FCI, CCSD(T))?

2.3 Quantum Error Correction (15 Questions)

Category: QEC & Surface Codes — HIGH PRIORITY

Q31. What is a stabilizer code? Define the $[[n, k, d]]$ notation.
Q32. Explain the surface code. Draw/describe the layout of data and ancilla qubits.
Q33. What is the threshold theorem for fault-tolerant quantum computing?
Q34. What is the difference between error detection and error correction?
Q35. What is a logical qubit vs. a physical qubit? Why do we need so many physical qubits?
Q36. What is lattice surgery? How is it used to perform logical CNOT gates?
Q37. What is magic state distillation and why is it the bottleneck for FTQC?
Q38. Describe the Minimum Weight Perfect Matching (MWPM) decoder.
Q39. What are Floquet codes? How do they differ from standard stabilizer codes?
Q40. What is the difference between topological codes and concatenated codes?
Q41. Explain the concept of code distance and how it relates to error suppression.
Q42. What is a syndrome measurement? How is it performed without collapsing the logical state?
Q43. What is transversal gate implementation? Why can't all gates be transversal?
Q44. How do qubit connectivity constraints affect QEC implementation?
Q45. What are LDPC codes and why are they promising for reducing QEC overhead?

2.4 Computational Chemistry on Quantum Computers (15 Questions)

Category: Quantum Chemistry — HIGH PRIORITY

Q46. Describe the end-to-end pipeline for simulating a molecule on a quantum computer.
Q47. What is chemical accuracy? Why is ∼1.6 mHa the target?
Q48. Explain second quantization. Why is it the natural language for quantum chemistry?
Q49. What is an active space? How do you decide which orbitals to include?
Q50. What is the difference between static and dynamic electron correlation?
Q51. How does basis set choice (STO-3G vs. cc-pVDZ) affect simulation accuracy and qubit count?
Q52. What are the main classical methods for electron correlation (CI, CC, DMRG) and their limitations?
Q53. Explain the Born-Oppenheimer approximation and when it breaks down.
Q54. How would you simulate excited states on a quantum computer?
Q55. What is the Hamiltonian simulation problem? Describe Trotterization.
Q56. What is qubitization and how does it improve over Trotter-based methods?
Q57. Explain quantum signal processing (QSP) and its role in modern quantum algorithms.
Q58. What molecules/materials are the most promising targets for quantum advantage?
Q59. How does error mitigation (ZNE, PEC) differ from error correction?
Q60. Describe Microsoft's hybrid HPC+quantum+AI chemistry demonstration.

2.5 Microsoft-Specific Questions (15 Questions)

Category: Microsoft Quantum — MUST KNOW

Q61. What is a topological qubit? How does it differ from superconducting/trapped-ion qubits?
Q62. Explain Majorana Zero Modes (MZMs) and non-Abelian anyons.
Q63. What is a topoconductor? What materials are used?
Q64. What is the Majorana 1 chip? What milestone does it represent?
Q65. Describe Microsoft's quantum computing roadmap (Level 1 → 2 → 3, six milestones).
Q66. What is the Azure Quantum Resource Estimator? Why is it important?
Q67. How does the tetron architecture work? Why 4 MZMs per qubit?
Q68. What is Azure Quantum Elements? How does it combine HPC, AI, and quantum?
Q69. Explain how Microsoft's measurement-based approach differs from traditional gate-based QC.
Q70. What are the key results from the MS+Quantinuum 12-logical-qubit experiment?
Q71. How do Microsoft's custom Floquet codes reduce QEC overhead compared to standard surface codes?
Q72. What is the DARPA US2QC program and where does Microsoft stand?
Q73. What is Q# and how does it differ from Python-based quantum frameworks?
Q74. What role does the Resource Estimator play in guiding algorithm development for FTQC?
Q75. How does Microsoft's approach to quantum computing complement your D-Wave experience?

2.6 Circuit-Based QC & Algorithms (10 Questions)

Category: Gate-Based Quantum Computing

Q76. What is a universal gate set? Give an example and explain why it's universal.
Q77. Explain Quantum Phase Estimation (QPE). Draw the circuit.
Q78. What is the Quantum Fourier Transform? How many gates does it require for n qubits?
Q79. Describe Grover's search algorithm and its quadratic speedup.
Q80. What is Hamiltonian simulation and why is it considered the "killer app" of QC?
Q81. Explain the Solovay-Kitaev theorem and gate synthesis.
Q82. What is measurement-based quantum computing (MBQC)? How does it work?
Q83. What is the Clifford+T gate set and why is it important for fault-tolerant QC?
Q84. Explain the concept of circuit depth and why it matters for NISQ devices.
Q85. What is the difference between coherent and incoherent noise?

2.7 Your Background & Fit (10 Questions)

Category: Personal & Behavioral

Q86. Tell me about your research at CERN with D-Wave. What problem did you solve?
Q87. How does your D-Wave quantum annealing experience translate to gate-based QC?
Q88. Describe your transmon qubit simulation work (pySPaRTA). What physics did you model?
Q89. How would you approach simulating a molecule's ground state energy, given your background?
Q90. What is the most challenging technical problem you've solved and how?
Q91. Why Microsoft Research? Why this specific quantum applications role?
Q92. What would you want to work on during the 12-week internship?
Q93. How do you handle a situation where your research direction leads to negative results?
Q94. Describe a time you collaborated across disciplines (physics, CS, engineering).
Q95. What questions do you have for us? (Prepare 5+ thoughtful questions.)

Part II

Study Material & Crash Courses

3 VQE (Variational Quantum Eigensolver)

3.1 Core Concept

VQE uses the **variational principle** to find the ground state energy:

$$E_0 \leq \langle \psi(\boldsymbol{\theta}) | \, H \, | \psi(\boldsymbol{\theta}) \rangle \tag{1}$$

The algorithm:

1. **Prepare** a parameterized trial state $|\psi(\boldsymbol{\theta})\rangle$ on a quantum computer
2. **Measure** the expectation value $\langle H \rangle = \langle \psi(\boldsymbol{\theta}) | \, H \, | \psi(\boldsymbol{\theta}) \rangle$
3. **Optimize** parameters $\boldsymbol{\theta}$ classically to minimize $\langle H \rangle$
4. **Repeat** until convergence

3.2 Hamiltonian Encoding

Second quantization: Express the molecular Hamiltonian using creation/annihilation operators:

$$H = \sum_{pq} h_{pq} \, a_p^\dagger a_q + \frac{1}{2} \sum_{pqrs} h_{pqrs} \, a_p^\dagger a_q^\dagger a_r a_s \tag{2}$$

Jordan-Wigner transformation: Maps fermionic operators to Pauli operators:

$$a_j^\dagger \to \frac{1}{2}(X_j - iY_j) \otimes Z_{j-1} \otimes \cdots \otimes Z_0 \tag{3}$$

This preserves anti-commutation relations but requires $O(N)$ Pauli weight.

Bravyi-Kitaev transformation: Alternative mapping with $O(\log N)$ Pauli weight.

Result: Hamiltonian as weighted sum of Pauli strings: $H = \sum_i c_i P_i$.

3.3 Ansatz Types

- **Hardware-efficient ansatz:** Layers of parameterized single-qubit rotations + entangling gates. Easy to implement but may suffer from barren plateaus.
- **UCCSD (Unitary Coupled Cluster Singles and Doubles):**

$$|\psi\rangle = e^{T - T^\dagger} \, |\mathrm{HF}\rangle \, , \quad T = T_1 + T_2 \tag{4}$$

  Chemically motivated; good accuracy but deep circuits.
- **ADAPT-VQE:** Grows the ansatz operator-by-operator based on gradient magnitudes.

3.4 VQE Code Example (PennyLane)

Listing 1: VQE for H_2 with PennyLane

```python
import pennylane as qml
from pennylane import numpy as np

# Load molecular data for H2
dataset = qml.data.load("qchem", molname="H2",
                        basis="STO-3G", bondlength=0.742)[0]
H = dataset.hamiltonian
hf_state = dataset.hf_state  # [1, 1, 0, 0]
qubits = len(hf_state)

dev = qml.device("default.qubit", wires=qubits)

@qml.qnode(dev)
def circuit(params):
    qml.BasisState(hf_state, wires=range(qubits))
    qml.DoubleExcitation(params[0], wires=[0, 1, 2, 3])
    return qml.expval(H)

# Optimize
opt = qml.GradientDescentOptimizer(stepsize=0.4)
params = np.array([0.0])
for step in range(100):
    params = opt.step(circuit, params)
    if step % 20 == 0:
        print(f"Step {step}: E = {circuit(params):.6f} Ha")
# Converges to ~ -1.13726 Ha (exact: -1.1373 Ha)
```

3.5 VQE vs QPE Comparison

Feature	VQE	QPE
Circuit depth	Shallow	Deep
Error tolerance	NISQ-friendly	Requires fault tolerance
Accuracy	Limited by ansatz	Exponentially precise
Classical cost	High (measurements + optimization)	Low post-processing
Best for	Near-term devices	Future FTQC

Table 1: VQE vs QPE comparison.

4 Quantum Error Correction & Surface Codes

4.1 Why QEC?

Physical error rates ~ 0.1–1%. Useful computation needs logical error rates $\sim 10^{-10}$. QEC encodes logical qubits into many physical qubits, enabling error detection and correction without collapsing the encoded state.

4.2 Stabilizer Formalism

For an $[[n, k, d]]$ code:

- n = physical qubits, k = logical qubits, d = code distance

- Code space = +1 eigenspace of commuting Pauli stabilizer operators
- Syndrome measurement identifies errors without revealing the logical state

4.3 The Surface Code

- Qubits on edges of a 2D lattice; X-stabilizers on vertices, Z-stabilizers on plaquettes
- $[[L^2 + (L-1)^2, 1, L]]$ code for a planar patch with distance L
- **Threshold:** $\sim 1\%$ circuit-level error rate (achievable today!)
- Only nearest-neighbor interactions required
- Logical qubit at 10^{-14} error rate needs $\sim 10^3$–10^4 physical qubits

4.4 Logical Operations

Operation	**Method**	**Cost**
Pauli X, Z	Transversal	Cheap
CNOT	Lattice surgery	Moderate
Hadamard, S	Code deformation	Moderate
T gate	Magic state distillation	**Expensive (bottleneck)**

Table 2: Logical gate implementations on surface codes.

4.5 Magic State Distillation

Non-Clifford gates (like T) cannot be performed transversally (Eastin–Knill theorem):

1. Prepare noisy magic states $|T\rangle = (|0\rangle + e^{i\pi/4}|1\rangle)/\sqrt{2}$
2. Distill to high fidelity using Clifford operations
3. Consume distilled states via gate teleportation

This is the **dominant overhead** in fault-tolerant quantum computing.

4.6 Beyond Surface Codes

- **Color codes:** Transversal Clifford gates, higher rate
- **LDPC codes:** Constant rate, less overhead, non-local connectivity
- **Floquet codes:** Error correction via periodic measurements (honeycomb code)
- **Microsoft's 4D codes:** Novel family reducing overhead vs. surface codes

5 Computational Chemistry on Quantum Computers

5.1 The Pipeline

$$\boxed{\text{Molecule}} \rightarrow \boxed{\begin{array}{c}\text{Classical}\\ \text{Pre-processing}\end{array}} \rightarrow \boxed{\begin{array}{c}\text{Quantum}\\ \text{Algorithm}\end{array}} \rightarrow \boxed{\begin{array}{c}\text{Post-}\\ \text{processing}\end{array}}$$

1. **Classical pre-processing:** Basis set selection, Hartree-Fock calculation, active space identification

2. **Fermion-to-qubit mapping:** Jordan-Wigner, Bravyi-Kitaev, parity mapping, qubit tapering
3. **Quantum algorithm:** VQE (NISQ) or QPE (FTQC)
4. **Post-processing:** Energy extraction, error mitigation, property calculation

5.2 Key Concepts

- **Chemical accuracy:** 1 kcal/mol $\approx$ 1.6 mHa — precision needed for meaningful predictions
- **Correlation energy:** $E_{\text{exact}} - E_{\text{HF}}$; the part quantum computers help compute
- **Active space:** Orbitals with variable occupation; $N_{\text{qubits}} = 2 \times N_{\text{active orbitals}}$
- **Classical shadows:** Efficient measurement protocol for many expectation values

5.3 MS+Quantinuum Chemistry Demonstration (Sep 2024)

Landmark Result: First HPC + Quantum + AI Chemistry Simulation

1. **HPC (Azure):** AutoCAS + AutoRXN identified active space of PNNP iron catalyst
2. **Quantum (Quantinuum H1):** 12 logical qubits prepared ground state
3. **AI:** Classical shadows trained AI model on quantum properties
4. **Result:** Ground state energy within chemical accuracy (1.6 mHa)
5. **Key finding:** Logical qubits outperformed physical qubits with 97% probability

6 Circuit-Based QC Fundamentals

6.1 Annealing vs. Gates

Feature	Quantum Annealing	Gate-Based QC
Model	Adiabatic evolution	Circuit model
Operations	Continuous time evolution	Discrete gates
Problem type	Optimization (QUBO/Ising)	Universal computation
Error correction	Limited	Full QEC possible
Universality	Not universal	Universal
Your experience	**Strong**	**Learning**

6.2 Essential Gates

Single-qubit: X, Y, Z (Pauli); H (Hadamard); S, T (phase); $R_x(\theta)$, $R_y(\theta)$, $R_z(\theta)$ (rotations).

Two-qubit: CNOT, CZ, SWAP, $\sqrt{i\text{SWAP}}$.

Universal sets: $\{H, T, \text{CNOT}\}$; $\{\text{Clifford} + T\}$ (standard fault-tolerant set).

6.3 Key Algorithms

1. **QPE:** Eigenvalue estimation; foundation of quantum chemistry ($O(1/\varepsilon)$ depth)
2. **QFT:** Quantum Fourier Transform; sub-component of QPE, Shor's
3. **Grover:** Quadratic speedup for unstructured search ($O(\sqrt{N})$ vs. $O(N)$)
4. **Hamiltonian simulation:** e^{-iHt} via Trotter-Suzuki, qubitization, QSP

7 Microsoft Quantum Stack

7.1 Roadmap

Level	Name	Description	Status
1	Foundational	Noisy physical qubits	Industry-wide
2	Resilient	Reliable logical qubits	**Active**
3	Scale	Quantum supercomputers	Future goal

Six milestones:

- ✓ Create & control Majoranas (May 2023)
- ✓ Hardware-protected topological qubit — **Majorana 1** (Feb 2025)
- ○ QEC demonstrations (in progress)
- ○ Fault-tolerant prototype (DARPA-funded)
- ○ Scalable FTQC
- ○ Quantum supercomputer (>1M reliable qubits, >1 rQOPS)

7.2 Majorana 1 — The Topological Qubit

- **Topoconductors:** InAs (semiconductor) + Al (superconductor), cooled to near absolute zero
- **MZMs:** Quasiparticles at nanowire ends; information stored as parity
- **Tetron:** 2 parallel nanowires + perpendicular superconductor = 4 MZMs per qubit
- **Operations:** All via measurements (digital, not analog)
- **Custom Floquet codes:** ∼10× QEC overhead reduction
- **Target:** 1 million qubits on a single chip

7.3 Azure Quantum Platform

- Q# programming language
- Quantum Development Kit (QDK)
- Azure Quantum Workspace
- Hardware partners (Quantinuum, IonQ, PASQAL, Rigetti)
- **Resource Estimator** (free, no Azure account needed)
- **Azure Quantum Elements** (HPC + AI + quantum for chemistry)

7.4 Key People

Person	Role	Focus
Krysta Svore	Technical Fellow, VP	Algorithms, logical qubits, chemistry
Chetan Nayak	Technical Fellow, CVP	Hardware, Majorana 1
Matthias Troyer	Technical Fellow	Algorithms, computational physics

Part III

7-Day Study Plan

Your Strengths → Gaps

Strengths: D-Wave annealing, PennyLane, transmon simulation, optimization, benchmarking.
Gaps: VQE, QEC/surface codes, computational chemistry, circuit-based QC, MS stack.

Day 1: VQE Foundations — HIGH PRIORITY

AM Read Tilly et al. Sections I–III. Watch PennyLane VQE video.

PM Hands-on: PennyLane VQE + molecular Hamiltonians tutorials. Run H_2, LiH.

Eve Read PennyLane chemistry docs. Notes on UCCSD, active space, measurement.

Day 2: Circuit-Based QC Fundamentals

AM PennyLane Codebook: Intro to QC, single-qubit gates.

PM PennyLane Codebook: Multi-qubit circuits, QPE module.

Eve Watch: "Quantum Computing for Computer Scientists" (MS Research, Helwer). Map annealing intuition to gates.

Day 3: QEC & Surface Codes — HIGH PRIORITY

AM Read Fowler et al. Sections 1–4. Quantum Katas: QEC module.

PM PennyLane Codebook: QEC. Study lattice surgery, magic state distillation.

Eve Read MS Floquet codes + 4D codes blog posts.

Day 4: Microsoft Quantum Stack — HIGH PRIORITY

AM Read: Majorana 1 blog. Listen: MS Podcast (Chetan Nayak). Study: MZMs, tetron.

PM Set up QDK. Run Resource Estimator chemistry tutorial.

Eve Read MS+Quantinuum chemistry blogs. Prepare HPC+quantum+AI talking points.

Day 5: Computational Chemistry Deep Dive

AM Read Cao et al. (near-term algorithms sections). Study BK, qubit tapering.

PM Implement VQE for H_2O (active space reduction). Explore ADAPT-VQE.

Eve Review MS chemistry simulation paper. Prepare molecular simulation discussion.

Day 6: Advanced Topics & Practice

AM Hamiltonian simulation (Trotter-Suzuki). Classical shadows protocol.

PM Practice all interview questions (Section 2). Prepare 5-min research pitch.

Eve Error mitigation (ZNE, PEC). Latest MS quantum blog posts.

Day 7: Review & Mock Interview

AM Review all notes. Re-read key blogs/papers.

PM Mock interview with colleague: research pitch, whiteboard VQE, QEC explanation, MS discussion.

Eve Prepare questions to ask interviewer. Rest.

Part IV

Comprehensive Answer Key

This section provides detailed model answers for all 95 interview questions. Study these to build deep understanding, but express answers in your own words during the interview.

8 Quantum Fundamentals — Answers

A1. Qubit, Superposition, and Classical Bits

A classical bit takes a definite value 0 or 1. A **qubit** is a two-level quantum system whose state is a superposition:

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle\,, \quad |\alpha|^2 + |\beta|^2 = 1.$$

Superposition means the qubit exists in a linear combination of $|0\rangle$ and $|1\rangle$ simultaneously. Upon measurement in the computational basis, it collapses to $|0\rangle$ with probability $|\alpha|^2$ and $|1\rangle$ with probability $|\beta|^2$. Key distinction: this is fundamentally different from classical probabilistic uncertainty — interference between amplitudes enables quantum algorithms.

A2. Entanglement and Bell States

Entanglement is a quantum correlation between two or more qubits that cannot be described by individual qubit states. An entangled state cannot be written as a tensor product $|\psi_A\rangle \otimes |\psi_B\rangle$. The four **Bell states** form a maximally entangled basis for two qubits:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle), \quad |\Phi^-\rangle = \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle) \tag{5}$$

$$|\Psi^+\rangle = \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle), \quad |\Psi^-\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle) \tag{6}$$

Measuring one qubit instantly determines the other's outcome, regardless of distance (violates Bell inequalities). This is the resource behind quantum teleportation, superdense coding, and entanglement-based QKD.

A3. Bloch Sphere

Any single-qubit pure state can be written as:

$$|\psi\rangle = \cos\frac{\theta}{2}\,|0\rangle + e^{i\phi}\sin\frac{\theta}{2}\,|1\rangle$$

This maps to a point on the unit sphere (Bloch sphere) with polar angle θ and azimuthal angle ϕ. $|0\rangle$ is the north pole, $|1\rangle$ the south pole, and $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$ lies on the equator. Single-qubit gates correspond to rotations: $R_x(\theta)$, $R_y(\theta)$, $R_z(\theta)$ rotate around the respective axes. Mixed states lie *inside* the sphere (closer to center = more mixed).

A4. No-Cloning Theorem

The **no-cloning theorem** states that it is impossible to create an identical copy of an arbitrary unknown quantum state. Formally, there is no unitary U such that $U|\psi\rangle|0\rangle = |\psi\rangle|\psi\rangle$ for all $|\psi\rangle$.

Proof sketch: Suppose $U|\psi\rangle|0\rangle = |\psi\rangle|\psi\rangle$ and $U|\phi\rangle|0\rangle = |\phi\rangle|\phi\rangle$. Taking the inner product: $\langle\psi|\phi\rangle = (\langle\psi|\phi\rangle)^2$, which is only satisfied when $\langle\psi|\phi\rangle = 0$ or 1. So cloning can only work for orthogonal or identical states.

Implications: (1) Enables quantum cryptography (eavesdroppers cannot copy quantum states). (2) Makes QEC harder — cannot simply copy qubits for redundancy; must use entanglement-based encoding. (3) Classical error correction (majority voting) doesn't directly apply.

A5. T_1 and T_2 Relaxation Times

These characterize decoherence in qubits:

- T_1 (**energy relaxation time**): Time for the qubit to decay from $|1\rangle$ to $|0\rangle$ (amplitude damping). Sets the maximum computation time.
- T_2 (**dephasing time**): Time for loss of phase coherence between $|0\rangle$ and $|1\rangle$ (phase damping). Always $T_2 \leq 2T_1$.
- T_2^*: Effective dephasing including inhomogeneous broadening; $T_2^* \leq T_2$.

Impact: Gate operations must be much faster than T_1, T_2. For superconducting qubits, typical values are $T_1 \sim 50$–$300\,\mu$s and $T_2 \sim 50$–$200\,\mu$s, while gate times are ~ 20–100 ns, allowing $\sim 10^3$–10^4 gates per coherence time.

Connection to your work: In your pySPaRTA transmon simulations, participation ratios directly affect T_1 through the relation $1/T_1 = \sum_i p_i/Q_i$ where p_i is the participation ratio and Q_i the quality factor of each lossy interface.

A6. Gate-Based vs. Adiabatic/Annealing QC

Gate-based QC:

- Computation via sequence of discrete unitary gates applied to qubits
- Universal: can implement any quantum algorithm
- Full QEC possible (surface codes, etc.)
- Examples: IBM, Google, Quantinuum, IonQ

Adiabatic/Annealing QC:

- Computation via continuous time evolution: start in ground state of simple H_i, slowly evolve to complex H_f whose ground state encodes the solution
- Adiabatic QC is theoretically universal (but impractical without gap guarantees)
- Quantum annealing (D-Wave) is a heuristic version without gap guarantees
- Limited error correction; naturally suited for optimization (QUBO/Ising)

Connection: The adiabatic theorem guarantees the system stays in the ground state if the evolution is slow compared to the minimum spectral gap Δ. Run time scales as $O(1/\Delta^2)$. In quantum annealing, this gap can be exponentially small for hard instances.

Bridge from your experience: Both VQE and annealing are fundamentally optimization problems. In annealing, the Hamiltonian encodes the cost function directly. In VQE, the ansatz defines the search space and a classical optimizer navigates the energy landscape. Your D-Wave intuition about energy landscapes and problem embedding translates to understanding cost landscapes in VQE.

A7. Adiabatic Theorem and Failure Conditions

The **adiabatic theorem** states: if a quantum system starts in the ground state of $H(0)$ and $H(t)$ evolves slowly enough, the system remains in the instantaneous ground state of $H(t)$.
Quantitatively: The evolution time must satisfy $T \gg \hbar \max_t \frac{|\langle 1(t)|\dot{H}(t)|0(t)\rangle|}{\Delta(t)^2}$ where $\Delta(t)$ is the energy gap.
Failure conditions: (1) **Small spectral gap** — first-order phase transitions create exponentially small gaps, requiring exponentially long annealing times. (2) **Too-fast evolution** — diabatic transitions cause Landau-Zener tunneling to excited states. (3) **Thermal excitation** — finite temperature allows transitions across the gap. (4) **Scaling problems** — for NP-hard problems, the gap often closes exponentially with problem size.
These are exactly the challenges you encountered in your D-Wave work — some QUBO instances have small gaps that lead to poor solution quality.

A8. Quantum Decoherence

Decoherence is the loss of quantum coherence due to interaction with the environment. The qubit becomes entangled with environmental degrees of freedom, effectively "measuring" the qubit.
Main decoherence channels:

- **Amplitude damping** (T_1): Energy exchange with environment; $|1\rangle \to |0\rangle$
- **Phase damping** (T_2): Loss of phase information; off-diagonal density matrix elements decay
- **Depolarizing noise**: Qubit randomly replaced by maximally mixed state with probability p
- **Bit-flip/phase-flip**: Pauli errors X, Z applied with some probability

For superconducting qubits (your expertise): Key decoherence sources include dielectric loss at material interfaces, quasiparticle tunneling, flux noise, and photon shot noise. Your pySPaRTA work models participation ratios at these lossy interfaces, which directly determines T_1.

A9. Density Matrix

The **density matrix** (density operator) ρ generalizes the state vector:

$$\rho = \sum_i p_i |\psi_i\rangle \langle\psi_i|$$

where p_i are classical probabilities and $|\psi_i\rangle$ are pure states.
Properties: ρ is Hermitian, positive semi-definite, $\mathrm{Tr}(\rho) = 1$. A pure state has $\mathrm{Tr}(\rho^2) = 1$; a mixed state has $\mathrm{Tr}(\rho^2) < 1$.
When to use: (1) Open quantum systems (decoherence). (2) Subsystems of entangled states (partial trace). (3) Ensemble averages. (4) Quantum channels and error models ($\rho \to \sum_k E_k \rho E_k^\dagger$, Kraus representation). Essential for QEC where you need to describe noisy qubits.

A10. Gottesman-Knill Theorem

The **Gottesman-Knill theorem** states that a quantum circuit consisting only of Clifford gates (H, S, CNOT), Pauli measurements, and computational-basis state preparation can be efficiently simulated classically in polynomial time using the stabilizer formalism.
Implication: Clifford circuits alone cannot provide quantum speedup. To achieve it, you need **non-Clifford gates** (like T gate). This is why the T gate (and magic state distillation) is so important in FTQC — it's the "ingredient" that makes quantum computation classically intractable.
Connection to QEC: Surface code operations that are "cheap" (transversal, lattice surgery) are exactly the Clifford operations. The expensive T gate via magic state distillation is needed precisely because it's the non-Clifford element required for universality.

A11. Pure vs. Mixed States

Pure state: Described by a single state vector $|\psi\rangle$. Maximum knowledge of the system. $\rho = |\psi\rangle\langle\psi|$, $\mathrm{Tr}(\rho^2) = 1$. Represents a single point on the Bloch sphere surface.
Mixed state: Statistical ensemble of pure states. $\rho = \sum_i p_i |\psi_i\rangle\langle\psi_i|$ with $\sum_i p_i = 1$. Cannot be described by a single state vector. $\mathrm{Tr}(\rho^2) < 1$. Lies inside the Bloch sphere.
Sources of mixedness: Decoherence, tracing out part of an entangled system, or classical ignorance of preparation. The maximally mixed state $\rho = I/2$ is at the center of the Bloch sphere.

A12. Quantum Teleportation

Protocol: Transfer an unknown qubit state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ from Alice to Bob using shared entanglement and classical communication:

1. Alice and Bob share a Bell state $|\Phi^+\rangle$
2. Alice performs a Bell measurement on her qubit + the state to teleport
3. Alice sends 2 classical bits (measurement outcome) to Bob
4. Bob applies a corrective Pauli operation based on the classical bits

No FTL communication: Bob's correction requires classical bits (limited by speed of light). Without the classical message, Bob's qubit looks like a random state. **No cloning:** Alice's original state is destroyed during Bell measurement.
Relevance to FTQC: Gate teleportation uses this idea to implement non-Clifford gates — prepare a magic state, teleport through it, and the gate is applied. This is central to magic state distillation.

A13. Quantum Zeno Effect

The **quantum Zeno effect**: Frequent measurements can "freeze" a quantum system's evolution. If a state $|\psi\rangle$ evolves under Hamiltonian H and is measured repeatedly at intervals Δt, the probability of finding it in a different state after each measurement decreases as $\Delta t \to 0$. For short times: $P(\text{transition}) \approx (\Delta E \cdot \Delta t/\hbar)^2$, which goes to zero quadratically. In the limit of continuous measurement, the system never evolves away from its initial state.
Relevance: The Zeno effect is the physical principle underlying quantum error detection — repeated syndrome measurements keep projecting the system back into the code space, preventing errors from accumulating.

A14. Ising Model and QUBO

The **Ising Hamiltonian** (familiar from your D-Wave work):

$$H_{\text{Ising}} = \sum_{i<j} J_{ij}\sigma_i^z\sigma_j^z + \sum_i h_i\sigma_i^z$$

QUBO (Quadratic Unconstrained Binary Optimization):

$$\min_{\mathbf{x}} \ \mathbf{x}^T Q\mathbf{x}, \quad x_i \in \{0,1\}$$

These are equivalent via the transformation $\sigma_i^z = 1 - 2x_i$. D-Wave natively solves the Ising model; your crop rotation MILP was mapped to QUBO for the D-Wave QPU.
Connection to VQE: The molecular Hamiltonian after Jordan-Wigner transformation becomes a weighted sum of Pauli strings (including Z-strings similar to Ising terms). VQE optimizes over this landscape — your QUBO embedding experience informs understanding of how physical problems map to qubit systems.

A15. Tensor Products and Hilbert Space

For a system of n qubits, the total Hilbert space is:

$$\mathcal{H} = \mathcal{H}_1 \otimes \mathcal{H}_2 \otimes \cdots \otimes \mathcal{H}_n, \quad \dim(\mathcal{H}) = 2^n$$

A product state $|\psi_1\rangle \otimes |\psi_2\rangle$ means the qubits are independent. An entangled state (like a Bell state) *cannot* be written as a tensor product.
The exponential growth 2^n is both the source of quantum computational power (exponentially many amplitudes) and the reason classical simulation is hard. This is precisely why molecular electronic structure is hard classically: N spin-orbitals $\to 2^N$ Slater determinants, but a quantum computer naturally represents this space with N qubits.

9 VQE & Variational Algorithms — Answers

A16. VQE and the Variational Principle

The **variational principle**: for any trial state $|\psi(\boldsymbol{\theta})\rangle$, its energy expectation value is an upper bound on the ground state energy:

$$E_0 \leq E(\boldsymbol{\theta}) = \langle\psi(\boldsymbol{\theta})| H |\psi(\boldsymbol{\theta})\rangle$$

VQE algorithm:

1. Choose an ansatz $|\psi(\boldsymbol{\theta})\rangle$ (parameterized quantum circuit)
2. Prepare the state on a quantum computer
3. Measure $\langle H\rangle$ by decomposing $H = \sum_i c_i P_i$ into Pauli strings and measuring each
4. Classical optimizer updates $\boldsymbol{\theta}$ to minimize $E(\boldsymbol{\theta})$
5. Repeat until convergence

Advantages: Shallow circuits (NISQ-compatible), noise-resilient (energy is variationally bounded), leverages classical optimization. **Disadvantages:** Not guaranteed to reach true ground state (depends on ansatz expressibility), barren plateaus, many measurements needed, classical optimization can get stuck in local minima.

A17. Parameterized Quantum Circuit and UCCSD

A **parameterized quantum circuit** (PQC) / ansatz: a circuit with adjustable parameters $\boldsymbol{\theta}$ that defines a family of quantum states. The circuit structure ("layout") is fixed; only the rotation angles change during optimization.

UCCSD (Unitary Coupled Cluster Singles and Doubles):

$$|\psi_{\text{UCCSD}}\rangle = e^{T_1+T_2-T_1^\dagger-T_2^\dagger}\,|\text{HF}\rangle$$

where:

- $T_1 = \sum_{ia} t_i^a a_a^\dagger a_i$ (single excitations: occupied $i \to$ virtual a)
- $T_2 = \sum_{ijab} t_{ij}^{ab} a_a^\dagger a_b^\dagger a_j a_i$ (double excitations)
- $|\text{HF}\rangle$ is the Hartree-Fock reference state

The cluster amplitudes t_i^a, t_{ij}^{ab} become the variational parameters. UCCSD is chemically motivated and systematically improvable, but requires $O(N^4)$ parameters and deep circuits after Trotterization.

A18. Jordan-Wigner Transformation — Worked Example

The **Jordan-Wigner (JW) transformation** maps fermionic creation/annihilation operators to qubit (Pauli) operators:

$$a_j^\dagger = \frac{1}{2}(X_j - iY_j) \otimes \prod_{k<j} Z_k, \quad a_j = \frac{1}{2}(X_j + iY_j) \otimes \prod_{k<j} Z_k$$

The Z-string enforces fermionic anti-commutation: $\{a_i, a_j^\dagger\} = \delta_{ij}$.

Example: For a 2-site system with Hamiltonian $H = t(a_0^\dagger a_1 + a_1^\dagger a_0)$:

$$a_0^\dagger a_1 = \frac{1}{4}(X_0 - iY_0)(X_1 + iY_1)Z_0 \tag{7}$$

$$= \frac{1}{4}(X_0 Z_0 X_1 + iX_0 Z_0 Y_1 - iY_0 Z_0 X_1 + Y_0 Z_0 Y_1) \tag{8}$$

Using $X_0 Z_0 = -iY_0$ and $Y_0 Z_0 = iX_0$:

$$H = \frac{t}{2}(X_0 X_1 + Y_0 Y_1)$$

This is a sum of 2 Pauli strings — each can be measured on the quantum computer.

A19. Jordan-Wigner vs. Bravyi-Kitaev

- **JW:** Each qubit stores *occupation* of one orbital. Pauli weight (number of non-identity terms) is $O(N)$ due to the Z-string. Simple to understand but creates long-range correlations.
- **BK:** Each qubit stores *partial parity* information. Pauli weight is $O(\log N)$. More complex mapping but results in more local Hamiltonian terms.
- **Parity mapping:** Each qubit stores parity of orbitals up to that index. Allows qubit reduction using symmetries.

When to prefer BK: Large systems where measurement overhead matters (fewer Pauli terms per operator). **When to prefer JW:** Small systems, pedagogical clarity, when occupation-number intuition is useful. For near-term VQE on small molecules, the difference is often minor.

A20. VQE vs. QPE

VQE: Hybrid quantum-classical. Uses shallow circuits + classical optimization. Energy has variational upper bound. Limited by ansatz expressibility, measurement noise, and optimization landscape. Best for NISQ devices.
QPE: Fully quantum. Uses deep circuits with controlled-U operations and QFT. Gives ground state energy to arbitrary precision ε with circuit depth $O(1/\varepsilon)$. Requires fault-tolerant hardware.
Transition regime (relevant to this role): "Early fault-tolerant" algorithms aim to bridge VQE and QPE. Examples include:

- **Quantum subspace expansion (QSE):** Run VQE then expand to nearby states
- **Quantum-classical hybrid QPE:** Use error mitigation + shorter QPE circuits
- **Randomized QPE:** Reduces circuit depth at the cost of shots

Microsoft's focus is exactly this transition — "early fault-tolerant" algorithms that work with $O(100)$ logical qubits.

A21. Active Space Selection

The **active space** is the subset of molecular orbitals treated with full quantum correlation:

- **Core orbitals:** Always doubly occupied (frozen)
- **Active orbitals:** Variable occupation (simulated on QC)
- **Virtual orbitals:** Always empty (excluded)

Selection criteria:

1. **Chemical intuition:** Include orbitals near the Fermi level, frontier orbitals (HOMO, LUMO)
2. **Occupation numbers:** From Hartree-Fock or MP2 natural orbitals; significantly fractionally occupied orbitals should be active
3. **AutoCAS:** Automated active space selection using entropy-based measures (used by Microsoft!)
4. **Standard notation:** CAS(ne, mo) = n electrons in m orbitals $\to$ $2m$ qubits

A22. Barren Plateaus

Barren plateaus: Phenomenon where the gradient of the cost function vanishes exponentially with system size for certain ansätze:

$$\mathrm{Var}\left[\frac{\partial E}{\partial \theta_i}\right] \sim O(2^{-n})$$

Causes:

- Deep, randomly initialized hardware-efficient ansätze form approximate 2-designs
- Global cost functions (measuring all qubits) vs. local cost functions
- High entanglement across the system
- Noise-induced barren plateaus (even shallow circuits in the presence of noise)

Mitigation strategies:

1. Use chemically-informed ansätze (UCCSD) instead of hardware-efficient ones
2. Layer-wise training (gradually increase circuit depth)
3. Identity initialization (start near $U = I$)
4. Local cost functions when possible
5. ADAPT-VQE (grows circuit systematically)

A23. Parameter-Shift Rule

The **parameter-shift rule** computes exact gradients on quantum hardware. For a gate of the form $e^{-i\theta G/2}$ where $G^2 = I$ (generator with eigenvalues ± 1):

$$\frac{\partial E}{\partial \theta} = \frac{E(\theta + \pi/2) - E(\theta - \pi/2)}{2}$$

How it works: Evaluate the circuit at two shifted parameter values and subtract. No classical approximation needed (unlike finite differences); the gradient is exact up to shot noise.
Cost: 2 circuit evaluations per parameter per gradient component $\to 2|\boldsymbol{\theta}|$ evaluations per gradient. For UCCSD with $O(N^4)$ parameters, this is expensive but embarrassingly parallel. Generalizes to gates with more eigenvalues using more shift points.

A24. ADAPT-VQE

ADAPT-VQE (Adaptive Derivative-Assembled Pseudo-Trotter VQE) builds the ansatz iteratively:

1. Start with a pool of operators $\{A_k\}$ (e.g., all fermionic excitation operators)
2. At each iteration, compute $\left.\frac{\partial E}{\partial \theta_k}\right|_{\theta_k=0}$ for each unused operator
3. Add the operator with the largest gradient to the ansatz
4. Re-optimize all parameters
5. Repeat until all gradients are below a threshold

Advantages: (1) Systematically builds only the necessary circuit complexity. (2) Avoids barren plateaus of random ansätze. (3) Often achieves chemical accuracy with fewer parameters than UCCSD. **Disadvantage:** Many gradient evaluations per iteration; requires large operator pool evaluation.

A25. Pauli Term Scaling

A molecular Hamiltonian in second quantization has $O(N^4)$ two-electron integrals for N spin-orbitals. After fermion-to-qubit mapping:

- **Jordan-Wigner:** $O(N^4)$ Pauli strings, each with weight up to $O(N)$
- **Bravyi-Kitaev:** $O(N^4)$ Pauli strings, each with weight up to $O(\log N)$

Each Pauli string requires separate measurement (modulo commuting groupings). The number of measurements for precision ε scales as $O(\sum_i |c_i|^2/\varepsilon^2)$ for individual Pauli measurements, or can be reduced via techniques like grouping commuting terms or using classical shadows.

A26. Classical Shadows

Classical shadows (Huang, Kueng, Preskill 2020): An efficient protocol for predicting many properties from few measurements.
Protocol:

1. Apply a random unitary U (drawn from an ensemble, e.g., random Clifford) to ρ
2. Measure in computational basis $\to$ outcome $\hat{b}$
3. Store classical snapshot $\hat{\rho} = \mathcal{M}^{-1}(U^\dagger |\hat{b}\rangle \langle\hat{b}| U)$
4. Average over many snapshots to predict $\mathrm{Tr}(O\rho)$ for any observable O

Power: $O(\log M/\varepsilon^2)$ measurements suffice to predict M observables to precision ε (exponentially better than measuring each separately). Used in the MS+Quantinuum chemistry demo to extract many molecular properties from their logical qubits.

A27. Hartree-Fock State

The **Hartree-Fock (HF) state** is the best single-determinant approximation to the ground state. Each electron occupies one spin-orbital; the wavefunction is a single Slater determinant.

Why start from HF in VQE:

1. Captures ∼99% of the total energy (mean-field level)
2. Provides natural orbital ordering for active space selection
3. Simple to prepare on a quantum computer: $|\mathrm{HF}\rangle = |1 \cdots 1\, 0 \cdots 0\rangle$ (occupied orbitals = 1)
4. VQE then captures the remaining **correlation energy** ($E_{\mathrm{corr}} = E_{\mathrm{exact}} - E_{\mathrm{HF}}$)

The correlation energy, despite being ∼1% of total energy, determines chemical properties like bond breaking, reaction barriers, and molecular geometries.

A28. Qubit Tapering

Qubit tapering reduces the number of qubits by exploiting symmetries of the molecular Hamiltonian:

1. Find $\mathbb{Z}_2$ symmetries of H (operators that commute with H and have eigenvalues ± 1)
2. Common symmetries: electron number parity, spin parity (S_z conservation), point group symmetries
3. Each independent symmetry removes 1 qubit
4. Typically saves 2–4 qubits for small molecules

Example: H_2 in minimal basis needs 4 qubits (4 spin-orbitals), but after tapering electron number and S_z symmetries, can be reduced to 2 qubits.

A29. Quantum Subspace Expansion (QSE)

QSE extends VQE to compute excited states and improve ground state estimates:

1. Run VQE to get approximate ground state $|\psi_0\rangle$
2. Generate a subspace $\{O_k |\psi_0\rangle\}$ using a set of operators (e.g., excitation operators)
3. Measure the Hamiltonian and overlap matrices in this subspace
4. Solve the generalized eigenvalue problem classically: $H\mathbf{c} = ES\mathbf{c}$

Advantages: Accesses excited states, corrects VQE errors, uses same circuit as VQE. Can also be combined with error mitigation.

A30. Benchmarking VQE vs. Classical

Classical baselines to compare against:

- **FCI (Full Configuration Interaction):** Exact within basis set. Exponential scaling $O(2^N)$. Gold standard for small systems.
- **CCSD(T):** "Gold standard" of practical quantum chemistry. Polynomial scaling $O(N^7)$ but breaks down for strongly correlated systems.
- **DMRG:** Excellent for 1D-like correlations. Polynomial in bond dimension.
- **AFQMC:** Auxiliary-field QMC. Good for weakly correlated, sign-problem-free systems.

Benchmarking protocol:

1. Compare energies (absolute and relative to FCI)
2. Check chemical accuracy (<1.6 mHa deviation from FCI)
3. Compare resource costs (qubits, gates, measurements, wall-clock time)
4. Test scaling behavior with system size

Your experience: Your D-Wave vs. Gurobi benchmarking methodology transfers directly.

10 Quantum Error Correction — Answers

A31. Stabilizer Codes and $[[n,k,d]]$ Notation

A **stabilizer code** is defined by a group $\mathcal{S}$ of commuting Pauli operators (stabilizers). The code space is the simultaneous $+1$ eigenspace of all stabilizers.

$[[n, k, d]]$ **notation:**

- n: number of physical qubits
- k: number of logical qubits ($k = n - |\text{generators of } \mathcal{S}|$)
- d: code distance = minimum weight of a logical operator (undetectable error)

The code can detect errors of weight up to $d - 1$ and correct errors of weight up to $\lfloor (d-1)/2 \rfloor$.

Examples:

- $[[3, 1, 1]]$ bit-flip code (3 physical qubits, 1 logical, distance 1)
- $[[5, 1, 3]]$ perfect code (smallest QEC code, corrects any single-qubit error)
- $[[7, 1, 3]]$ Steane code (CSS code, transversal Clifford gates)
- Surface code: $[[L^2 + (L-1)^2, 1, L]]$

A32. Surface Code Layout

The **surface code** uses a 2D lattice:

- **Data qubits:** On edges of the lattice
- **X-type stabilizers (star operators):** $A_v = \prod_{e \ni v} X_e$ for each vertex v
- **Z-type stabilizers (plaquette operators):** $B_p = \prod_{e \in p} Z_e$ for each face p
- All stabilizers commute: $[A_v, B_p] = 0$ (each shares 0 or 2 edges)

Logical operators:

- $\bar{X}$: chain of X operators connecting top and bottom boundaries
- $\bar{Z}$: chain of Z operators connecting left and right boundaries

Minimum-length chains have weight L (the code distance), so errors of weight $< L/2$ don't cause logical errors. Typical for distance-L surface code: $\sim 2L^2$ physical qubits per logical qubit.

A33. Threshold Theorem

The **threshold theorem**: If the physical error rate p is below a threshold p_{th}, then by increasing the code distance L, the logical error rate can be reduced to arbitrarily low values:

$$p_L \approx C \left(\frac{p}{p_{\text{th}}} \right)^{\lfloor (L+1)/2 \rfloor}$$

Surface code thresholds:

- Phenomenological noise: $p_{\text{th}} \approx 3\%$
- Circuit-level noise: $p_{\text{th}} \approx 1\%$
- Depolarizing with MWPM: $p_{\text{th}} \approx 0.57\%$

Implication: Below threshold, every doubling of code distance *squares* the logical error rate ($p/p_{\text{th}} < 1 \Rightarrow$ exponential suppression). Current hardware (Google Sycamore, IBM, Quantinuum) is approaching or crossing threshold, making FTQC a engineering challenge rather than a fundamental physics one.

A34. Error Detection vs. Error Correction

Error detection: Identify that an error occurred, but not which error. Action: discard the corrupted state and try again (post-selection). Requires $d \geq 2$ (detect weight-1 errors).
Error correction: Identify which error occurred and actively fix it. Requires $d \geq 3$ (correct weight-1 errors). Uses syndrome $\rightarrow$ decoder $\rightarrow$ correction.
Key distinction: Detection is "cheaper" (fewer physical qubits) but wastes shots. Correction is more resource-intensive but preserves all successful outcomes. In practice, early FTQC may use detection (smaller codes) before scaling to full correction.
Microsoft's Milestone 3 targets QEC *demonstrations* — likely detection first, then correction, with their topological qubits.

A35. Logical vs. Physical Qubits

Physical qubit: The actual hardware element (transmon, trapped ion, topological). Subject to noise, decoherence, gate errors.
Logical qubit: An error-corrected qubit encoded across many physical qubits. Protected by QEC; much lower effective error rate.
Why so many physical qubits? For a distance-L surface code:

- $\sim 2L^2$ physical qubits per logical qubit
- To achieve $p_L \sim 10^{-10}$ with $p_{\text{phys}} = 10^{-3}$: need $L \sim 17$, so ~ 578 physical qubits per logical qubit
- Plus T-factories for magic state distillation (each factory occupies many logical qubits)

Example: A practical quantum chemistry simulation might need ~ 100 logical qubits $\times \sim 1000$ physical each $= \sim 100{,}000$ physical qubits. This is why resource estimation matters.

A36. Lattice Surgery

Lattice surgery performs logical operations between surface code patches by temporarily merging and splitting their boundaries:
Logical CNOT via lattice surgery:

1. **Merge:** Extend the boundary of two adjacent patches to create shared stabilizers. This measures the product of their logical operators (e.g., $\bar{Z}_1\bar{Z}_2$).
2. **Split:** Separate the patches, projecting into an eigenstate of the measured operator.
3. Combine with Pauli corrections based on measurement outcomes.

Advantages: Only requires local operations (nearest-neighbor). Time cost: $O(d)$ QEC rounds per surgery operation. Space cost: temporary "routing space" between patches. This is the primary method for logical gates in all current FTQC proposals, including Microsoft's.

A37. Magic State Distillation — Bottleneck

Why it's needed: The Eastin-Knill theorem proves no QEC code can implement a universal gate set transversally. For the surface code, Clifford gates are "easy" (transversal/lattice surgery) but the T gate requires magic states.
Distillation protocol:

1. Prepare n noisy copies of $|T\rangle = (|0\rangle + e^{i\pi/4}|1\rangle)/\sqrt{2}$ with error rate p
2. Use a distillation circuit (Clifford operations) to produce k cleaner copies with error $O(p^t)$
3. Common protocol: 15-to-1 distillation ($15 \rightarrow 1$ magic state, error $p \rightarrow 35p^3$)

Why it's the bottleneck:

- Each T gate consumes one distilled magic state
- Distillation factories occupy many logical qubits ($\sim$1000s of physical qubits each)
- Algorithms may need millions of T gates, requiring many factories running in parallel
- Microsoft's Floquet codes (on topological qubits) reduce this overhead $\sim 10\times$

A38. MWPM Decoder

The syndrome of a surface code identifies the *endpoints* of error chains but not the chains themselves. The **Minimum Weight Perfect Matching (MWPM)** decoder:

1. Parse syndrome: identify defects (violated stabilizers)
2. Construct a complete graph where nodes are defects and edge weights are Manhattan distances
3. Find the minimum-weight perfect matching (pairs defects by shortest total distance)
4. Apply correction along matched paths

Performance: Near-optimal for independent errors. Time complexity $O(n^3)$ (Edmonds' algorithm), which can be a bottleneck for real-time decoding. **Alternatives:** Union-Find decoder ($O(n\alpha(n))$, nearly linear), machine learning decoders, belief propagation.

A39. Floquet Codes

Floquet codes perform error correction through periodically scheduled measurements rather than measuring a fixed set of stabilizers simultaneously:

- Measurements involve only 2-body operators (simpler than 4-body surface code stabilizers)
- The code space is defined dynamically by the measurement sequence
- **Honeycomb code:** Prominent example on a hexagonal lattice with 2-body checks

Microsoft's approach: Custom Floquet codes designed for the tetron qubit geometry. Because topological qubits have inherently lower error rates (hardware protection), the Floquet codes can achieve the same logical error rate with fewer physical qubits — roughly $\sim$10$\times$ fewer than standard surface codes on conventional qubits.

A40. Topological vs. Concatenated Codes

Topological codes (surface, color, toric):

- Define stabilizers on a lattice; errors correspond to strings/chains
- Only need local (nearest-neighbor) interactions
- High threshold ($\sim$1%), making them practical for 2D hardware
- Single logical qubit per patch

Concatenated codes (e.g., Steane code [[7,1,3]] concatenated recursively):

- Encode a logical qubit, then re-encode each physical qubit into another code
- L levels of concatenation: error rate $p \to (p/p_{\text{th}})^{2^L}$ (doubly exponential suppression)
- Requires long-range connectivity
- Lower threshold ($\sim 10^{-4}$ for non-topological concatenation)

LDPC codes offer a third approach: constant-rate encoding (many logical qubits per block) with potentially much lower overhead, but require non-local connectivity.

A41–A45. Remaining QEC Answers (Summary)

A41 (Code distance): d is the minimum weight of any undetectable error (logical operator). Logical error rate suppressed as $(p/p_{\text{th}})^{d/2}$. Doubling d squares the logical error rate.
A42 (Syndrome measurement): Ancilla qubits are entangled with data qubits via CNOT gates, then measured. The data qubits' logical state is not disturbed because we only learn about parity (stabilizer eigenvalues), not the encoded information.
A43 (Transversal gates): A gate is transversal if it acts independently on corresponding qubits of two code blocks: $\bar{U} = U^{\otimes n}$. Errors cannot spread within a block. The Eastin-Knill theorem shows no code admits a universal transversal gate set.
A44 (Connectivity constraints): Surface codes need only nearest-neighbor connectivity (ideal for 2D chips). LDPC codes need non-local connections (challenging for planar fabrication). Connectivity determines which codes are practical for a given hardware.
A45 (LDPC codes): Low-Density Parity-Check codes encode $k = \Theta(n)$ logical qubits in n physical qubits (constant rate). Dramatically less overhead than surface codes (which encode only 1 logical qubit per $\sim 2d^2$ physical). But need non-local connectivity and have lower thresholds. Active research area at Microsoft and IBM.
Note: $\mathcal{S}$ denotes the stabilizer group.

11 Computational Chemistry — Answers

A46. End-to-End Molecular Simulation Pipeline

1. **Define molecule:** Atomic coordinates, charge, spin multiplicity
2. **Choose basis set:** STO-3G (minimal, few qubits), cc-pVDZ (accurate, more qubits)
3. **Run Hartree-Fock:** Get reference state and molecular integrals (PySCF)
4. **Select active space:** CAS(ne, mo) based on orbital analysis or AutoCAS
5. **Map to qubits:** JW or BK transformation; $2m$ qubits (+ possible tapering)
6. **Choose ansatz:** UCCSD for accuracy, hardware-efficient for low depth
7. **Run VQE (or QPE):** Quantum circuit execution + classical optimization
8. **Post-process:** Compare to classical baselines, compute molecular properties

In PennyLane: steps 1–5 are handled by `qml.qchem.molecular_hamiltonian()`.

A47. Chemical Accuracy

Chemical accuracy = 1 kcal/mol ≈ 1.6 mHa (milliHartree) ≈ 43 meV.
This is the precision needed for quantum chemistry predictions to be *chemically meaningful*: correctly predicting which product is thermodynamically favored, accurate reaction barriers, reliable molecular geometries. Below this threshold, quantum chemistry predictions become useful for drug design, catalyst optimization, and materials discovery.
The MS+Quantinuum chemistry demo achieved 1.6 mHa accuracy — right at the chemical accuracy threshold — using only 12 logical qubits. This is a proof-of-concept for the fault-tolerant regime.

A48. Second Quantization

Second quantization represents quantum states in terms of occupation numbers of single-particle orbitals, using creation ($a^\dagger$) and annihilation (a) operators.
For **fermions** (electrons), the anti-commutation relations are:

$$\{a_i, a_j^\dagger\} = \delta_{ij}, \quad \{a_i, a_j\} = \{a_i^\dagger, a_j^\dagger\} = 0$$

Why natural for quantum chemistry:

1. Automatically handles antisymmetry (Pauli exclusion)
2. Hamiltonian is compactly written with at most 2-body terms
3. Particle number is not fixed → can describe ionization, electron attachment
4. Maps naturally to qubits via JW/BK transformations (occupation numbers ↔ qubit states)

A49–A55. Remaining Chemistry Answers (Summary)

A49 (Active space): See A21. Key addition: for transition metal catalysts (like MS's PNNP example), active space must include d-orbitals and their bonding partners.
A50 (Static vs. dynamic correlation): *Static*: near-degenerate orbitals require multiple determinants (e.g., bond breaking). Captured by CASSCF/VQE. *Dynamic*: many small contributions from excited determinants. Captured by perturbation theory (MP2, CASPT2) or CC methods.
A51 (Basis set effects): STO-3G: 1 function/orbital, few qubits, poor accuracy. cc-pVDZ: better accuracy, ∼3× more qubits. Basis set error is often larger than correlation error for small bases.
A52 (Classical correlation methods): CI: exact in limit (FCI), exponential scaling. CC: size-extensive, polynomial scaling, gold standard CCSD(T) $O(N^7)$, fails for multi-reference. DMRG: tensor network, excellent for 1D-like correlations, MPS-based.
A53 (Born-Oppenheimer): Separates nuclear and electronic motion. Breaks down at conical intersections (excited-state dynamics, photochemistry).
A54 (Excited states): Methods: QSE, variational quantum deflation (VQD), equation-of-motion (EOM) approaches, multi-state VQE. Important for photochemistry, spectroscopy.
A55 (Trotterization): $e^{-i(A+B)t} \approx (e^{-iAt/n}e^{-iBt/n})^n$ with error $O(t^2/n)$. Higher-order: $O(t^{p+1}/n^p)$. Qubitization and QSP achieve optimal $O(t)$ scaling.

A56. Qubitization

Qubitization encodes the Hamiltonian into a unitary operator using block-encoding:

$$U = \begin{pmatrix} H/\lambda & \cdot \\ \cdot & \cdot \end{pmatrix}$$

where $\lambda = \sum_i |c_i|$ (1-norm of coefficients). Combined with quantum phase estimation, this gives eigenvalues of H with query complexity $O(\lambda t/\varepsilon)$ — optimal in t and ε.
Advantage over Trotter: No Trotter error accumulation; query complexity scales as $O(\lambda)$ rather than $O(\lambda^2)$. Enables more efficient fault-tolerant chemistry simulations. This is the algorithm family Microsoft's Resource Estimator often evaluates.

A57. Quantum Signal Processing (QSP)

QSP implements arbitrary polynomial transformations of a block-encoded Hamiltonian. Given a unitary signal U and a sequence of signal-processing angles $\{\phi_k\}$:

$$U_{\text{QSP}} = e^{i\phi_0 Z} \prod_{k=1}^{d} \left[U \cdot e^{i\phi_k Z} \right]$$

this achieves $P(H)$ for any polynomial P of degree d.
Why it matters: Unifies many quantum algorithms (QPE, Hamiltonian simulation, amplitude amplification) under one framework. Achieves optimal query complexity for ground-state energy estimation. The Quantum Singular Value Transformation (QSVT) generalizes QSP to matrix functions — considered the "grand unified theory" of quantum algorithms.

A58. Promising Molecules for Quantum Advantage

Strong candidates:

- **FeMoCo** (nitrogenase active site): biological nitrogen fixation; ∼100+ qubits needed; enormous industrial impact (Haber-Bosch process replacement)
- **Transition metal catalysts:** Strong static correlation in d-orbitals breaks CCSD(T). MS's PNNP catalyst demo targets this class.
- **High-T_c superconductors:** Hubbard model, strongly correlated; understanding mechanism could transform energy technology
- **Battery materials:** Li-ion cathode chemistry (multi-reference character during charge/discharge)
- **Carbon capture:** Metal-organic frameworks for CO_2 binding

Why these? All feature strong electron correlation where classical methods fail. The active spaces are large enough to be classically intractable but potentially addressable by near-term FTQC with $O(100\text{–}1000)$ logical qubits.

A59. Error Mitigation vs. Error Correction

Error mitigation (NISQ era):

- **Zero-Noise Extrapolation (ZNE):** Run at multiple noise levels, extrapolate to zero noise
- **Probabilistic Error Cancellation (PEC):** Decompose ideal operations into noisy ones + corrections; exponential sampling overhead
- **Symmetry verification:** Post-select on known symmetries (particle number, spin)
- No qubit overhead, but limited scalability; sampling cost grows with circuit size

Error correction (FTQC era):

- Active encoding, syndrome measurement, real-time correction
- Large qubit overhead but enables arbitrarily long computations
- Scalable: logical error rate decreases exponentially with code distance

Bridge: Early FTQC may combine both — use small codes (partial error correction) + mitigation for residual errors. This is exactly the "early fault-tolerant" regime of the CS-632 role.

A60. MS+Quantinuum Chemistry Demonstration

What they did (Sep 2024):

1. Used Azure HPC (AutoCAS) to automatically select the active space of a PNNP iron catalyst
2. AutoRXN identified the key reaction pathway
3. Prepared the ground state on Quantinuum H1 using **12 logical qubits** (encoded from ∼56 physical qubits)
4. Used **classical shadows** to efficiently extract molecular properties
5. Fed quantum-measured properties into an AI model to predict chemical behavior

Key results:

- Ground state energy within **chemical accuracy** (1.6 mHa)
- Logical qubits outperformed noisy physical qubits with **97% probability**
- First end-to-end **HPC + quantum + AI** chemistry workflow
- Demonstrated the Azure Quantum Elements pipeline

Why it matters for your interview: This is the template for what the CS-632 role will extend. Be ready to discuss how you would adapt this workflow to new molecules/materials.

12 Microsoft-Specific — Answers

A61. Topological Qubits

Topological qubits encode quantum information in global properties of a many-body system that are protected by topology, not just by energy gaps.

Key differences:

- **Superconducting (transmon):** Information in quantized charge/flux states of a Josephson junction. Sensitive to local noise (charge, flux, photon number fluctuations). You simulated these!
- **Trapped ion:** Information in internal electronic states. Low error rates but slow gates.
- **Topological (Microsoft):** Information in non-local degrees of freedom (parity of Majorana zero modes). Local perturbations cannot affect the qubit state → hardware-level protection.

Analogy: Like the difference between storing information on a single magnetic domain (vulnerable to local heating) vs. in the topology of a knot (can't be changed by local deformations without cutting).

A62. Majorana Zero Modes and Non-Abelian Anyons

Majorana Zero Modes (MZMs): Quasiparticles that appear at the ends of topological superconducting nanowires. They are their own antiparticle ($\gamma = \gamma^\dagger$) and have zero energy.

Non-Abelian anyons: Particles whose exchange (braiding) produces a unitary transformation on the degenerate ground state, rather than just a phase. Two MZMs at the ends of a wire share a single fermionic mode with occupation 0 or 1 (even/odd parity). Braiding MZMs implements topologically protected gates.

Why useful for QC: Braiding operations are topologically protected — small perturbations don't change the result. This gives inherent noise protection at the hardware level, reducing QEC overhead.

Microsoft's realization: InAs/Al hybrid nanowire devices (topoconductors) host MZMs at their ends. Majorana 1 demonstrated creation, control, and readout of these modes.

A63. Topoconductor

A **topoconductor** is Microsoft's name for a new class of materials that exhibits a topological superconducting phase, hosting Majorana zero modes.
Material: Indium arsenide (InAs) semiconductor nanowire + thin aluminum (Al) superconductor shell. When cooled to near absolute zero ($\sim$20 mK) and placed in a magnetic field, the InAs/Al interface enters a topological phase.
Key physics: Strong spin-orbit coupling in InAs + s-wave superconductivity from Al + Zeeman field creates the topological regime. The topological gap protects MZMs from local perturbations. Microsoft took $\sim$17 years and thousands of material science iterations to achieve reliably clean interfaces.

A64. Majorana 1

Majorana 1 (announced Feb 2025): World's first quantum processor powered by a Topological Core. Represents Microsoft's Milestone 2 achievement.
Key facts:

- 8 topological qubits on a single chip (tetron geometry)
- Each qubit: 4 MZMs on 2 parallel nanowires connected by a trivial superconductor
- All-digital control (measurement-based, not analog rotation)
- Initial measurement fidelity $\sim$99%
- Parity lifetime $\sim$1 ms (far exceeding measurement time)
- Demonstrated: qubit creation, manipulation, readout, idle coherence
- Published in Nature (peer-reviewed), with verification by DARPA testers

What it doesn't yet have: Two-qubit operations (braiding), QEC. These are Milestones 3–4.
Your angle: "Having simulated transmon qubits at the physical level, I'm fascinated by how topological protection fundamentally changes the error model and QEC requirements. The digital, measurement-based control of topological qubits is conceptually elegant."

A65–A75. Remaining Microsoft Answers (Summary)

A65 (Roadmap): Level 1 (noisy physical) → Level 2 (reliable logical) → Level 3 (quantum supercomputer). Six milestones: (1) ✓ Create Majoranas. (2) ✓ Topological qubit. (3) QEC demos. (4) FT prototype. (5) Scalable FTQC. (6) Quantum supercomputer.
A66 (Resource Estimator): Estimates physical qubits, runtime, T-factory count for FT algorithms. Supports surface code + custom codes. Free, browser-based. Critical tool for algorithm development — tells you whether an algorithm is practical before hardware exists.
A67 (Tetron): Two parallel nanowires forming a loop with 4 MZMs. Qubit encoded in parity of the loop. "Tetron" = four Majoranas. Operations via tunnel-coupling measurements between MZMs.
A68 (Azure Quantum Elements): Integrated platform: classical HPC (DFT, HF) + AI (surrogate models) + quantum (VQE/QPE on partner hardware). Workflow: AutoRXN (reaction pathways) → AutoCAS (active space) → quantum algorithm → AI postprocessing.
A69 (Measurement-based QC): All computation via measurements on entangled states. Topological qubits naturally fit MBQC because operations on MZMs are performed via parity measurements, not unitary gates.
A70 (12 logical qubits): Created on Quantinuum H1 using [[4,2,2]] codes. Demonstrated below-physical logical error rates. Combined with chemistry simulation.
A71 (Floquet codes overhead): 10x fewer physical qubits vs. standard surface codes, because topological qubits have lower base error rates and Floquet codes are tailored to the measurement-based architecture.
A72 (DARPA US2QC): Utility-Scale Quantum Computing program. Microsoft selected for final phase. Goal: demonstrate practical quantum utility. Validates Microsoft's approach at the government level.
A73 (Q#): Functional quantum programming language. Type-safe, integrated resource estimation. Differs from Python frameworks by being designed for compilation to FTQC hardware.
A74 (Resource Estimator role): Guides algorithm research by answering "how many qubits and how long?" Enables co-design between algorithms and hardware. Try the chemistry tutorial!
A75 (Bridge to your experience): Both D-Wave annealing and FTQC chemistry solve hard optimization/eigenvalue problems. Your benchmarking methodology (quantum vs. classical solvers), hybrid algorithm design, and hardware understanding transfer directly. Microsoft needs people who can assess practical performance — that's what you do.

13 Circuit-Based QC & Algorithms — Answers

A76–A85. Circuit QC Answers (Summary)

A76 (Universal gate set): $\{H, T, \text{CNOT}\}$ is universal. H creates superposition, CNOT entangles, T provides non-Clifford rotation. Solovay-Kitaev: any 1-qubit unitary can be approximated to precision ε using $O(\log^c(1/\varepsilon))$ gates from a universal set.
A77 (QPE): Uses n ancilla qubits, controlled-U^{2^k} operations, and inverse QFT to extract eigenphase ϕ of $U|\psi\rangle = e^{2\pi i\phi}|\psi\rangle$ to n-bit precision. For chemistry: $U = e^{-iHt}$ gives $\phi = Et/2\pi$ $\rightarrow$ energy eigenvalue.
A78 (QFT): Maps $|j\rangle \rightarrow \frac{1}{\sqrt{N}}\sum_k e^{2\pi ijk/N}|k\rangle$. Uses $O(n^2)$ gates (Hadamards + controlled-phase rotations). Key subroutine of QPE, Shor's.
A79 (Grover): Marks target states with oracle, amplifies amplitude via inversion about mean. $O(\sqrt{N})$ queries for N items. Quadratic speedup is provably optimal for unstructured search.
A80 (Hamiltonian simulation): "Nature isn't classical, dammit" (Feynman). Simulate e^{-iHt}. Trotter: $O(\lambda^2 t^2/\varepsilon)$ queries. Qubitization: $O(\lambda t)$. QSP/QSVT: optimal. "Killer app" because it's provably exponentially faster than any known classical method for strongly correlated systems.
A81 (Solovay-Kitaev): Any single-qubit gate can be approximated to error ε using $O(\log^c(1/\varepsilon))$ Clifford+T gates, with $c \approx 3.97$. Efficient classical algorithm to find the decomposition. Practical implication: T-count determines resource cost.
A82 (MBQC): Start with entangled cluster state. Perform single-qubit measurements in adaptive bases. Measurement outcomes + classical processing drive computation. Universal with 2D cluster states. Microsoft's natural paradigm: topological qubits are measured, not gated.
A83 (Clifford+T): Clifford gates (H, S, CNOT) + T gate forms a universal set. Clifford gates are "free" in FTQC (transversal/lattice surgery). T gate is the expensive resource (magic state distillation). Algorithm efficiency often measured by "T-count" and "T-depth".
A84 (Circuit depth): Total number of gate layers (sequential time steps). NISQ devices limited to ∼100s of layers (decoherence). FTQC can handle millions. Depth $\times$ error rate $\ll 1$ for reliable computation. VQE uses shallow circuits precisely to stay within NISQ depth limits.
A85 (Coherent vs. incoherent noise): *Coherent*: unitary errors (over-rotation, crosstalk). Preserve purity but can interfere constructively $\rightarrow$ worse than random. *Incoherent*: non-unitary (damping, dephasing, depolarizing). Described by quantum channels (Kraus operators). QEC is designed for incoherent noise; coherent errors must be randomized (twirling) first.

14 Background & Behavioral — Answers

A86. Your D-Wave/CERN Research

Model answer: "At the Open Quantum Institute at CERN, I developed a hybrid quantum-classical pipeline to solve a real-world sustainable food production optimization problem on the D-Wave Advantage quantum annealer.

The problem is a mixed-integer linear program (MILP) for optimal crop-to-farm allocation, optimizing nutritional yield and environmental sustainability scores across up to 100 farms. I formulated it as a QUBO, embedded it on D-Wave's Pegasus topology, and developed custom constraint-handling heuristics.

I comprehensively benchmarked quantum annealing against Gurobi (classical) across problem sizes from 3 to 100 farms, analyzing solution quality, constraint violation rates, and scaling behavior. Key findings: D-Wave showed competitive performance for small instances but struggled with constraint satisfaction at scale, motivating hybrid approaches.

This experience directly translates to the role: benchmarking quantum vs. classical, understanding hardware constraints, and designing hybrid algorithms are exactly what's needed for assessing chemistry applications on early FTQC devices."

A87. Annealing to Gate-Based Translation

Key parallels:

1. Both minimize energy/cost functions — annealing searches for ground states physically; VQE does it variationally
2. QUBO encoding $\leftrightarrow$ Hamiltonian encoding (Jordan-Wigner); both map problems to qubit representations
3. Embedding on QPU topology $\leftrightarrow$ circuit compilation and qubit routing
4. Penalty coefficients for constraints $\leftrightarrow$ Lagrange multipliers in VQE
5. My benchmarking methodology transfers directly to chemistry applications

Key differences I'm learning: Gate-based QC is universal, supports QEC, and enables precise eigenvalue extraction (QPE). These make it suitable for chemistry, which annealing cannot address.

A88. Transmon Simulation (pySPaRTA)

Model answer: "I developed pySPaRTA, a Python simulation suite for Ansys HFSS/Maxwell that models participation ratios in transmon qubits. Participation ratios quantify how much electromagnetic energy participates in lossy interfaces (metal-air, metal-substrate, substrate-air), directly determining T_1 coherence times.

The tool automates parameter sweeps over qubit geometry (junction dimensions, pad sizes, coupling strengths) and validates against experimental data from HF vapor-etched Al and Al/Nb transmons fabricated in our lab.

This gave me deep understanding of: superconducting qubit physics, decoherence mechanisms, electromagnetic simulation, and the gap between theoretical models and experimental reality — all of which inform how I think about the error models relevant to QEC and fault-tolerant computation."

A89. Approaching Ground State Simulation

"Drawing from both my hardware knowledge and optimization experience, I would:

1. Define the molecule and choose an appropriate basis set (start with STO-3G for prototyping, then improve)
2. Run Hartree-Fock classically (PySCF) and analyze orbital structure
3. Select an active space using natural orbital occupations (or AutoCAS for larger molecules)
4. Map to qubits via Jordan-Wigner; apply symmetry tapering to reduce qubit count
5. Choose ansatz: UCCSD for accuracy or hardware-efficient for near-term constraints
6. Run VQE with gradient-based optimization (parameter-shift rule)
7. Benchmark against FCI (if tractable) and CCSD(T)
8. Use the Azure Quantum Resource Estimator to assess what it would take to run this on fault-tolerant hardware

This combines my optimization skills (step 6), benchmarking experience (step 7), and growing chemistry knowledge (steps 1–5)."

A91. Why Microsoft Research?

"Three reasons: (1) Microsoft's topological approach is fundamentally different from other quantum platforms, and I find the physics beautiful — hardware-level protection changes the entire error-correction calculus. (2) The Azure Quantum Elements vision of combining HPC, quantum, and AI mirrors my own hybrid approach from D-Wave work at CERN. (3) Microsoft is uniquely positioned at the "early fault-tolerant" frontier, which is where the most impactful algorithm development will happen in the next 5–10 years — and that's exactly the focus of this internship."

A92. What to Work On During the Internship

"I'd love to contribute to extending the MS+Quantinuum chemistry demonstration to new molecular systems — particularly strongly correlated transition metal complexes relevant to catalysis or materials science. Specifically:

- Prototype early fault-tolerant algorithms (beyond VQE, toward QPE variants) for a target molecule
- Use the Resource Estimator to analyze the cost-benefit of different algorithmic choices
- Benchmark quantum-computed results against classical baselines (DMRG, CCSD(T))
- Explore how classical shadows can be leveraged to extract more molecular properties from fewer quantum measurements

My benchmarking experience and hybrid algorithm design skills would directly serve these goals."

Appendix: Complete Resource List

Microsoft Resources

Resource	URL
MS Research Quantum	https://www.microsoft.com/research/research-area/quantum-computing/
Quantum Roadmap	https://quantum.microsoft.com/en-us/our-story
Azure Quantum Overview	https://learn.microsoft.com/en-us/azure/quantum/overview-azure-quantum
Resource Estimator	https://learn.microsoft.com/en-us/azure/quantum/intro-to-resource-estimation
Quantum Katas	https://quantum.microsoft.com/en-us/tools/quantum-katas
MS Quantum Blog	https://azure.microsoft.com/en-us/blog/quantum/
Majorana 1 Blog	https://azure.microsoft.com/en-us/blog/quantum/2025/02/19/majorana-1/
MS+Quantinuum Chemistry	https://azure.microsoft.com/en-us/blog/quantum/2024/09/10/
Q# Quick Start	https://learn.microsoft.com/en-us/azure/quantum/qsharp-quickstart
Chemistry Resource Estimation	https://learn.microsoft.com/en-us/azure/quantum/tutorial-resource-estimator-chemistry
MS Podcast (Nayak)	https://aka.ms/MSRPodTopo
Azure Quantum Elements	https://quantum.microsoft.com/en-us/quantum-elements/product-overview

Learning Platforms

Resource	URL
PennyLane VQE Tutorial	https://pennylane.ai/qml/demos/tutorial_vqe/
PennyLane Molecular Hamiltonians	https://pennylane.ai/qml/demos/tutorial_quantum_chemistry/
PennyLane Chemistry Docs	https://docs.pennylane.ai/en/stable/introduction/chemistry.html
PennyLane Codebook	https://pennylane.ai/codebook
Qiskit Textbook	https://learning.quantum.ibm.com/catalog/courses

Key Papers

Paper	URL
Peruzzo et al. VQE (2014)	https://arxiv.org/abs/1304.3061
Tilly et al. VQE Review (2022)	https://arxiv.org/abs/2111.05176

Fowler et al. Surface Codes (2012)	https://arxiv.org/abs/1208.0928
Cao et al. Quantum Chemistry (2019)	https://pubs.acs.org/doi/10.1021/acs.chemrev.8b00803
MS Chemistry Simulation	http://aka.ms/ArXivMSFTChemSimPaper
Majorana 1 Nature Paper	https://aka.ms/MSQuantumNaturePaper
MS Logical Qubits Paper	http://aka.ms/ArXivMSFTLQPaper

Your Competitive Advantages

Interview Narrative

"I've worked with quantum hardware at both the physical level (transmon simulations, pySPaRTA) and the application level (hybrid quantum-classical optimization on D-Wave at CERN). VQE is fundamentally an optimization problem — my QUBO/MILP benchmarking experience translates directly. I'm now bridging into quantum chemistry, the highest-impact quantum application. Microsoft's approach — logical qubits + HPC + AI — mirrors the hybrid methods I've already deployed at scale."

Part V

Hands-On Coding Projects

Each project is designed to take **2–3 hours** and produces a concrete deliverable. Projects are ordered to follow the 7-day study plan and cover all major interview topic areas. Use **PennyLane** as the primary framework unless noted otherwise.

Setup

```
pip install pennylane pennylane-qchem matplotlib numpy scipy
```

Optional: `pip install qiskit qiskit-nature azure-quantum qsharp stim pymatching`

Note: PySCF is Linux/macOS only. Projects use PennyLane's pre-computed datasets (`qml.data.load()`) which work on all platforms.

A Project 1: VQE for H_2 — Full Pipeline Day 1

Concepts Reinforced: Q16–Q20, Q27, Q30 — VQE, Variational Principle, Ansatz, JW Transform

Duration: 2–3 hours **Framework:** PennyLane

Why This Matters

The **Variational Quantum Eigensolver (VQE)** is the most important near-term quantum algorithm for chemistry. It finds the ground state energy of molecules — the foundation of all chemical predictions. Understanding VQE deeply is essential for any quantum chemistry role.

Theory Background

The Variational Principle: For any trial wavefunction $|\psi\rangle$, the energy expectation value is always $\geq$ the true ground state energy:

$$E_0 \leq \langle\psi| H |\psi\rangle$$

This means if we parameterize $|\psi(\boldsymbol{\theta})\rangle$ and minimize $\langle H\rangle$, we approach E_0.

The VQE Loop:

1. A **quantum computer** prepares a parameterized state $|\psi(\boldsymbol{\theta})\rangle$
2. The quantum computer **measures** $\langle H\rangle$ by decomposing H into measurable Pauli strings
3. A **classical optimizer** updates $\boldsymbol{\theta}$ to reduce the energy
4. Repeat until convergence

The Jordan-Wigner Transformation: Electrons are fermions (obey anti-commutation). Qubits are not. We need a mapping:

$$a_j^\dagger \to \frac{1}{2}(X_j - iY_j) \cdot Z_{j-1} \cdot Z_{j-2} \cdots Z_0$$

The Z-string ensures fermionic anti-commutation is preserved. This converts the molecular Hamiltonian into a sum of Pauli strings that we can measure on a quantum computer.

Task

Build VQE for H_2 from scratch — understand every stage of the pipeline.

Step-by-Step Instructions

Step 1: Load molecular data

```
import pennylane as qml
from pennylane import numpy as np

# Load precomputed H2 data (no PySCF needed!)
dataset = qml.data.load("qchem", molname="H2",
                        basis="STO-3G", bondlength=0.742)[0]

H = dataset.hamiltonian        # The qubit Hamiltonian
hf_state = dataset.hf_state    # [1, 1, 0, 0] = 2 electrons in 2 orbitals
n_qubits = len(hf_state)       # 4 qubits for H2 in STO-3G
```

Step 2: Inspect the Hamiltonian

```
# Print all Pauli terms
print(f"Number of Pauli terms: {len(H.ops)}")
for coeff, op in zip(H.coeffs, H.ops):
    print(f"  {coeff:.6f} * {op}")
# You should see ~15 terms like: 0.17 * Z0, -0.22 * Z0Z1, etc.
```

Step 3: Setup the quantum device

```
dev = qml.device("default.qubit", wires=n_qubits)
```

Step 4: Build the VQE circuit with ansatz

The **ansatz** is the parameterized circuit that prepares our trial state. For chemistry, we use excitation gates that move electrons between orbitals.

```
@qml.qnode(dev)
def vqe_circuit(params):
    # Step 1: Prepare Hartree-Fock state |1100>
    # This puts 2 electrons in the lowest 2 orbitals
    qml.BasisState(hf_state, wires=range(n_qubits))

    # Step 2: Apply double excitation ansatz
    # This excites both electrons from orbitals 0,1 to 2,3
    # params[0] controls the amplitude of this excitation
    qml.DoubleExcitation(params[0], wires=[0, 1, 2, 3])

    # Step 3: Measure the Hamiltonian expectation value
    return qml.expval(H)
```

Step 5: Run the optimization

```
# Initialize parameter
params = np.array([0.0], requires_grad=True)

# Choose optimizer
opt = qml.GradientDescentOptimizer(stepsize=0.4)

# Store energies for plotting
energies = []

# Optimization loop
for step in range(100):
    params, energy = opt.step_and_cost(vqe_circuit, params)
    energies.append(energy)
```

```
    if step % 20 == 0:
        print(f"Step {step}: E = {energy:.6f} Ha")

print(f"Final VQE energy: {energies[-1]:.6f} Ha")
print(f"Exact FCI energy: -1.1373 Ha")
print(f"Error: {(energies[-1] - (-1.1373))*1000:.2f} mHa")
```

Step 6: Verify gradients with parameter-shift rule

The **parameter-shift rule** computes exact gradients on quantum hardware:

$$\frac{\partial E}{\partial \theta} = \frac{E(\theta + \pi/2) - E(\theta - \pi/2)}{2}$$

```
# Manual parameter-shift gradient
theta = params[0]
grad_param_shift = (vqe_circuit([theta + np.pi/2])
                    - vqe_circuit([theta - np.pi/2])) / 2

# Finite difference gradient (for comparison)
eps = 1e-5
grad_finite_diff = (vqe_circuit([theta + eps])
                    - vqe_circuit([theta - eps])) / (2*eps)

print(f"Parameter-shift gradient: {grad_param_shift:.8f}")
print(f"Finite-diff gradient:     {grad_finite_diff:.8f}")
print(f"Difference: {abs(grad_param_shift - grad_finite_diff):.2e}")
```

Deliverables

- ✓ Energy convergence plot (energy vs. iteration) — should converge smoothly
- ✓ Print: number of Pauli terms (∼15), final energy (≈ -1.137 Ha), error vs. FCI (< 1.6 mHa for chemical accuracy)
- ✓ Verify: parameter-shift gradient matches finite-difference to $< 10^{-6}$

Key Takeaways

- VQE is a **hybrid** algorithm: quantum state preparation + classical optimization
- The molecular Hamiltonian becomes a **sum of Pauli strings** via Jordan-Wigner
- The **ansatz choice** determines what states VQE can reach
- **Parameter-shift rule** gives exact gradients — key for quantum hardware

B Project 2: Potential Energy Surface & Bond Dissociation Day 1

Concepts Reinforced: Q47, Q51 — Chemical Accuracy, Basis Sets, Static Correlation

Duration: 2 hours **Framework:** PennyLane

Why This Matters

The **potential energy surface (PES)** shows how a molecule's energy changes with geometry. This is fundamental to chemistry: it predicts bond lengths, reaction barriers, and molecular stability. Understanding where classical methods fail (and quantum computers can help) is crucial.

Theory Background

Chemical Accuracy: The gold standard is 1 kcal/mol ≈ **1.6 mHa** (milliHartree). Below this threshold, predictions become chemically meaningful — you can trust reaction predictions, equilibrium geometries, etc.

Hartree-Fock Failure: HF uses a single Slater determinant (one electron configuration). This works well at equilibrium but fails catastrophically during **bond breaking** because multiple configurations become equally important. This is called **static correlation**.

Why Quantum Computers Help: VQE naturally captures multi-configurational character through its parameterized ansatz. Where HF fails, VQE succeeds.

Basis Sets: The basis set determines how accurately we represent electron orbitals:

- **STO-3G**: Minimal basis, 1 function per orbital, fewest qubits, poor accuracy
- **6-31G**: Split-valence, better accuracy, ∼2× more qubits
- **cc-pVDZ/TZ**: Correlation-consistent, even better, ∼4–8× more qubits

More qubits → better accuracy but harder to simulate on quantum hardware.

Task

Compute the H_2 dissociation curve and observe where Hartree-Fock fails.

Step-by-Step Instructions

Step 1: Set up the bond length sweep

```python
import pennylane as qml
from pennylane import numpy as np
import matplotlib.pyplot as plt

# Bond lengths to scan (Angstroms)
# Equilibrium is ~0.74 A, we go from compressed to stretched
bond_lengths = [0.5, 0.7, 0.742, 0.9, 1.1, 1.3, 1.5, 1.7, 2.1]

# Storage for results
hf_energies = []
fci_energies = []
vqe_energies = []
```

Step 2: Compute FCI (exact) energy via matrix diagonalization

Full Configuration Interaction (FCI) is the **exact answer** within a given basis set. We compute it by diagonalizing the Hamiltonian matrix.

```python
def compute_fci_energy(H):
    """Exact diagonalization of the Hamiltonian."""
    # Convert Hamiltonian to matrix form
    H_matrix = qml.matrix(H)
    # Find eigenvalues
    eigenvalues = np.linalg.eigvalsh(H_matrix)
```

```
    # Ground state is the smallest eigenvalue
    return np.min(eigenvalues)
```

Step 3: Run VQE at each bond length

```
for bl in bond_lengths:
    print(f"Bond length: {bl} A")

    # Load data for this geometry
    dataset = qml.data.load("qchem", molname="H2",
                            basis="STO-3G", bondlength=bl)[0]
    H = dataset.hamiltonian
    hf_state = dataset.hf_state
    n_qubits = len(hf_state)

    # Store HF energy (from dataset or compute)
    hf_energies.append(dataset.hf_energy)

    # Compute FCI energy
    fci_e = compute_fci_energy(H)
    fci_energies.append(fci_e)

    # Run VQE
    dev = qml.device("default.qubit", wires=n_qubits)

    @qml.qnode(dev)
    def circuit(params):
        qml.BasisState(hf_state, wires=range(n_qubits))
        qml.DoubleExcitation(params[0], wires=[0, 1, 2, 3])
        return qml.expval(H)

    # Optimize
    opt = qml.GradientDescentOptimizer(stepsize=0.4)
    params = np.array([0.0], requires_grad=True)
    for _ in range(50):
        params = opt.step(circuit, params)

    vqe_energies.append(circuit(params))
    print(f"  HF: {hf_energies[-1]:.4f}, FCI: {fci_e:.4f}, VQE: {
        vqe_energies[-1]:.4f}")
```

Step 4: Plot the results

```
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8, 8))

# Top panel: Energy curves
ax1.plot(bond_lengths, hf_energies, 'b-o', label='Hartree-Fock')
ax1.plot(bond_lengths, fci_energies, 'k-s', label='FCI (exact)')
ax1.plot(bond_lengths, vqe_energies, 'r-^', label='VQE')
ax1.set_xlabel('Bond Length (Angstrom)')
ax1.set_ylabel('Energy (Hartree)')
ax1.legend()
ax1.set_title('H2 Dissociation Curve')

# Bottom panel: Error vs FCI
vqe_errors = [(v - f)*1000 for v, f in zip(vqe_energies, fci_energies)]
hf_errors = [(h - f)*1000 for h, f in zip(hf_energies, fci_energies)]
ax2.plot(bond_lengths, hf_errors, 'b-o', label='HF error')
ax2.plot(bond_lengths, vqe_errors, 'r-^', label='VQE error')
ax2.axhline(y=1.6, color='g', linestyle='--', label='Chemical accuracy')
```

```
ax2.axhline(y=-1.6, color='g', linestyle='--')
ax2.set_xlabel('Bond Length (Angstrom)')
ax2.set_ylabel('Error vs FCI (mHa)')
ax2.legend()

plt.tight_layout()
plt.savefig('h2_dissociation.png', dpi=150)
plt.show()
```

Deliverables

- ✓ 2-panel figure: energy curves + error vs. FCI
- ✓ Observation: HF error grows with bond length (HF cannot describe bond breaking)
- ✓ Observation: VQE stays within or near chemical accuracy throughout
- ✓ Table: equilibrium bond length from each method (should all be ∼0.74 Å)

Key Takeaways

- **Hartree-Fock fails at stretched geometries** — this is static correlation
- **VQE captures multi-reference character** — the double excitation allows electron correlation
- **Chemical accuracy** (1.6 mHa) is the target for useful predictions
- Larger basis sets give better accuracy but require more qubits

C Project 3: Ansätze Comparison & Barren Plateaus Day 2

Concepts Reinforced: Q17, Q22, Q23, Q24 — Ansatz Design, Barren Plateaus, Gradients

Duration: 2–3 hours **Framework:** PennyLane

Why This Matters

The **ansatz** (parameterized circuit) determines what quantum states VQE can reach. A bad ansatz means VQE cannot find the ground state, no matter how long you optimize. Understanding ansatz design and the **barren plateau problem** is critical for practical quantum algorithms.

Theory Background

Types of Ansätze:

- **Hardware-efficient**: Random-looking circuits with rotation layers + entangling gates. Easy to implement but not physics-informed.
- **Chemically-inspired (UCCSD)**: Based on excitation operators from coupled-cluster theory. Harder to implement but physically motivated.

Barren Plateaus: For deep, random circuits, the gradient variance decays **exponentially** with qubit count:

$$\mathrm{Var}\left[\frac{\partial E}{\partial \theta}\right] \sim 2^{-n}$$

This means for 50 qubits, gradients are $\sim 10^{-15}$ in magnitude — effectively zero! Optimization becomes impossible because you cannot tell which direction to go.

Why This Happens: Random circuits spread amplitude uniformly over the exponentially large Hilbert space. Small parameter changes have exponentially small effects on the output.

Mitigation Strategies:

- Use **shallow** circuits (fewer layers)
- Use **structured** ansätze with physical motivation (UCCSD)
- Initialize parameters near identity (small rotations)
- Use **local** cost functions when possible

Task

Compare hardware-efficient vs. chemically-inspired ansätze. Empirically demonstrate the barren plateau phenomenon.

Step-by-Step Instructions

Step 1: Define the three ansatz types

```python
import pennylane as qml
from pennylane import numpy as np

# Load H2 data
dataset = qml.data.load("qchem", molname="H2",
                        basis="STO-3G", bondlength=0.742)[0]
H = dataset.hamiltonian
hf_state = dataset.hf_state
n_qubits = 4

dev = qml.device("default.qubit", wires=n_qubits)

# ANSATZ 1: Hardware-efficient (random structure)
def hardware_efficient_layer(params, wires):
    """One layer: Ry, Rz on each qubit, then CNOT ladder."""
    n = len(wires)
    for i, w in enumerate(wires):
        qml.RY(params[2*i], wires=w)
        qml.RZ(params[2*i+1], wires=w)
    for i in range(n-1):
        qml.CNOT(wires=[wires[i], wires[i+1]])

@qml.qnode(dev)
def ansatz_hardware_efficient(params, n_layers=2):
    # Start from |0000>
    for layer in range(n_layers):
        start = layer * 2 * n_qubits
        hardware_efficient_layer(params[start:start+2*n_qubits], range(
            n_qubits))
    return qml.expval(H)

# ANSATZ 2: UCCSD (single double excitation)
@qml.qnode(dev)
def ansatz_uccsd(params):
    qml.BasisState(hf_state, wires=range(n_qubits))
    qml.DoubleExcitation(params[0], wires=[0, 1, 2, 3])
```

```
    return qml.expval(H)

# ANSATZ 3: Extended UCCSD (singles + doubles)
@qml.qnode(dev)
def ansatz_extended(params):
    qml.BasisState(hf_state, wires=range(n_qubits))
    # Single excitations
    qml.SingleExcitation(params[0], wires=[0, 2])
    qml.SingleExcitation(params[1], wires=[1, 3])
    # Double excitation
    qml.DoubleExcitation(params[2], wires=[0, 1, 2, 3])
    return qml.expval(H)
```

Step 2: Compare VQE performance

```
def run_vqe(cost_fn, n_params, n_steps=100):
    """Run VQE and return final energy + convergence history."""
    opt = qml.GradientDescentOptimizer(stepsize=0.4)
    params = np.random.randn(n_params) * 0.1  # Small random init
    params = np.array(params, requires_grad=True)

    energies = []
    for _ in range(n_steps):
        params, energy = opt.step_and_cost(cost_fn, params)
        energies.append(energy)

    return energies[-1], energies

# Run each ansatz
e_hw, hist_hw = run_vqe(lambda p: ansatz_hardware_efficient(p, n_layers=2),
                        n_params=2*n_qubits*2)
e_uccsd, hist_uccsd = run_vqe(ansatz_uccsd, n_params=1)
e_ext, hist_ext = run_vqe(ansatz_extended, n_params=3)

print(f"Hardware-efficient: {e_hw:.6f} Ha")
print(f"UCCSD:              {e_uccsd:.6f} Ha")
print(f"Extended:           {e_ext:.6f} Ha")
print(f"Exact (FCI):        -1.1373 Ha")
```

Step 3: Barren plateau experiment

Now we demonstrate that hardware-efficient ansätze have exponentially vanishing gradients as qubit count increases.

```
def measure_gradient_variance(n_qubits, n_layers, n_samples=200):
    """Measure variance of gradient w.r.t. first parameter."""
    dev = qml.device("default.qubit", wires=n_qubits)
    n_params = 2 * n_qubits * n_layers

    # Simple cost function: just measure Z on first qubit
    @qml.qnode(dev)
    def circuit(params):
        for layer in range(n_layers):
            for i in range(n_qubits):
                qml.RY(params[layer*2*n_qubits + 2*i], wires=i)
                qml.RZ(params[layer*2*n_qubits + 2*i + 1], wires=i)
            for i in range(n_qubits - 1):
                qml.CNOT(wires=[i, i+1])
        return qml.expval(qml.PauliZ(0))
```

```python

    gradients = []
    for _ in range(n_samples):
        # Random parameters
        params = np.random.uniform(0, 2*np.pi, n_params)
        params = np.array(params, requires_grad=True)
        # Compute gradient of first parameter via parameter-shift
        grad = (circuit(params.at[0].set(params[0] + np.pi/2))
               - circuit(params.at[0].set(params[0] - np.pi/2))) / 2
        gradients.append(grad)

    return np.var(gradients)

# Sweep qubit counts
qubit_counts = [4, 6, 8, 10]
variances = []
for n in qubit_counts:
    var = measure_gradient_variance(n, n_layers=n)  # depth = width
    variances.append(var)
    print(f"n={n} qubits: Var[grad] = {var:.2e}")

# Plot
plt.semilogy(qubit_counts, variances, 'o-')
plt.xlabel('Number of qubits')
plt.ylabel('Gradient variance')
plt.title('Barren Plateau: Gradient Variance vs Qubits')
plt.savefig('barren_plateau.png')
```

Deliverables

- ✓ Table: final energy, # parameters, circuit depth for each ansatz
- ✓ Log-plot: gradient variance vs. qubit count (should show exponential decay)
- ✓ Fit: $\log(\text{Var}) = -\alpha n + c$ to extract decay rate α
- ✓ Observation: chemically-informed ansätze avoid barren plateaus because they're shallow and structured

Key Takeaways

- **Ansatz choice dramatically affects VQE success**
- **Hardware-efficient ansätze** are easy to implement but suffer from barren plateaus
- **Chemically-motivated ansätze** (UCCSD) are shallower and avoid the plateau problem
- This is why **ADAPT-VQE** (Project 12) builds ansätze adaptively — to use only necessary operators

D Project 4: Quantum Phase Estimation from Scratch Day 2

Concepts Reinforced: Q77, Q78, Q80 — QPE, QFT, Hamiltonian Simulation

Duration: 2–3 hours **Framework:** PennyLane

Why This Matters

Quantum Phase Estimation (QPE) is the workhorse algorithm for fault-tolerant quantum chemistry. Unlike VQE (which approximates), QPE extracts eigenvalues with exponential precision. Understanding QPE is essential for the "post-NISQ" era that Microsoft is targeting.

Theory Background

The Problem: Given a unitary U and one of its eigenstates $|u\rangle$, where $U|u\rangle = e^{2\pi i\phi}|u\rangle$, find the **phase** ϕ.

Why This Solves Chemistry: For a molecular Hamiltonian H, we can construct $U = e^{-iHt}$. The eigenstates of U are the same as H, with phases $\phi = Et/2\pi$. So finding ϕ gives us the energy E!

The QPE Circuit:

1. Prepare n **ancilla qubits** in $|+\rangle^{\otimes n}$ (uniform superposition)
2. Prepare the **eigenstate** $|u\rangle$ in the system register
3. Apply **controlled-U^{2^k}** operations: ancilla k controls U^{2^k}
4. Apply **inverse QFT** to the ancilla register
5. Measure ancillas — the result is ϕ in binary!

Precision: With n ancilla qubits, we get n bits of precision: $\Delta\phi = 2^{-n}$. More ancillas = more precision.

The Quantum Fourier Transform (QFT):

$$\text{QFT} : |j\rangle \to \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{2\pi i jk/2^n} |k\rangle$$

It uses Hadamard gates and controlled-phase rotations. The inverse QFT "decodes" the phase information.

Task

Implement QPE from scratch and use it to find eigenvalues.

Step-by-Step Instructions

Step 1: Implement the QFT

```python
import pennylane as qml
from pennylane import numpy as np

def qft(wires):
    """Quantum Fourier Transform on given wires."""
    n = len(wires)
    for i in range(n):
        qml.Hadamard(wires=wires[i])
        for j in range(i+1, n):
            # Controlled phase rotation
            angle = np.pi / (2 ** (j - i))
            qml.ControlledPhaseShift(angle, wires=[wires[j], wires[i]])
    # Swap to reverse bit order
    for i in range(n // 2):
        qml.SWAP(wires=[wires[i], wires[n-1-i]])
```

```python

def inverse_qft(wires):
    """Inverse QFT - just reverse the operations."""
    qml.adjoint(qft)(wires)
```

Step 2: Test QFT with a simple example

```python
# Test: QFT should convert computational basis to Fourier basis
dev = qml.device("default.qubit", wires=3)

@qml.qnode(dev)
def test_qft():
    qml.BasisState([1, 0, 1], wires=[0,1,2])  # |5> = |101>
    qft(wires=[0,1,2])
    return qml.state()

state = test_qft()
# Should see uniform amplitudes with phases encoding "5"
```

Step 3: Implement QPE for a simple unitary

Start with $U = R_z(\phi)$ where we know ϕ.

```python
n_ancilla = 3
n_system = 1
total_wires = n_ancilla + n_system

dev = qml.device("default.qubit", wires=total_wires)

# The unitary: Rz(2*pi*phase)
# Eigenstate |1> has eigenvalue e^{i*phase*2*pi}
true_phase = 3/8  # We'll try to recover this

@qml.qnode(dev)
def qpe_circuit():
    # 1. Prepare ancillas in |+>
    for i in range(n_ancilla):
        qml.Hadamard(wires=i)

    # 2. Prepare eigenstate |1> on system qubit
    qml.PauliX(wires=n_ancilla)

    # 3. Controlled-U^(2^k) operations
    for k in range(n_ancilla):
        power = 2 ** (n_ancilla - 1 - k)
        angle = 2 * np.pi * true_phase * power
        # Controlled-Rz = controlled phase on |1>
        qml.ControlledPhaseShift(angle, wires=[k, n_ancilla])

    # 4. Inverse QFT on ancillas
    inverse_qft(wires=range(n_ancilla))

    # 5. Measure ancillas
    return qml.probs(wires=range(n_ancilla))

probs = qpe_circuit()
estimated_phase = np.argmax(probs) / (2 ** n_ancilla)
print(f"True phase: {true_phase}")
print(f"Estimated phase: {estimated_phase}")
print(f"Error: {abs(true_phase - estimated_phase)}")
```

Step 4: Apply QPE to a Hamiltonian

For a simple 2-qubit Ising model, implement Trotterized time evolution and extract the ground state energy.

```
# Ising Hamiltonian: H = J*Z0*Z1 + h*(X0 + X1)
J, h = 1.0, 0.5

def trotter_step(t, n_steps):
    """First-order Trotter approximation to exp(-i*H*t)."""
    dt = t / n_steps
    for _ in range(n_steps):
        # exp(-i * J * Z0*Z1 * dt)
        qml.IsingZZ(2*J*dt, wires=[0, 1])
        # exp(-i * h * X0 * dt) and exp(-i * h * X1 * dt)
        qml.RX(2*h*dt, wires=0)
        qml.RX(2*h*dt, wires=1)

# Exact ground state energy (for comparison)
H_matrix = J * np.array([[1,0,0,0],[0,-1,0,0],[0,0,-1,0],[0,0,0,1]])
H_matrix += h * np.kron(np.array([[0,1],[1,0]]), np.eye(2))
H_matrix += h * np.kron(np.eye(2), np.array([[0,1],[1,0]]))
exact_energies = np.linalg.eigvalsh(H_matrix)
print(f"Exact ground state energy: {min(exact_energies):.4f}")
```

Deliverables

- ✓ Working QFT implementation with verification test
- ✓ QPE successfully extracts $\phi = 3/8$ exactly with 3 ancillas
- ✓ Plot: histogram of QPE measurement outcomes showing peak at correct phase
- ✓ Comparison: energy precision vs. number of ancilla qubits

Key Takeaways

- **QPE gives exponentially precise eigenvalues** with circuit depth $O(1/\varepsilon)$
- **QFT is the key subroutine** that decodes phase information
- **Trotterization** approximates e^{-iHt} for any Hamiltonian
- QPE requires **fault-tolerant hardware** (deep circuits) — this is Microsoft's target

E Project 5: Quantum Error Correction Simulation — Day 3

Concepts Reinforced: Q31–Q35, Q38, Q41, Q42 — Stabilizer Codes, Syndrome, Threshold

Duration: 2–3 hours **Framework:** PennyLane (or `stim` + `pymatching`)

Why This Matters

Quantum Error Correction (QEC) is what makes fault-tolerant quantum computing possible. Without it, errors accumulate and destroy quantum states before useful computation finishes. Understanding QEC is essential for the Microsoft role, which targets "early fault-tolerant" devices.

Theory Background

The Problem: Physical qubits have error rates $p \sim 0.1$–1%. Useful quantum chemistry needs error rates $< 10^{-10}$. That's a gap of 8–10 orders of magnitude!

The Solution: Encode one **logical qubit** into many **physical qubits**. Errors on physical qubits can be detected and corrected without destroying the encoded information.

Stabilizer Codes: Define a code by its **stabilizer generators** — Pauli operators whose $+1$ eigenspace is the code space. Measuring stabilizers gives us a **syndrome** that reveals errors without revealing the encoded information.

The Repetition Code (simplest example):

- Encode: $|0\rangle_L = |000\rangle$, $|1\rangle_L = |111\rangle$
- Stabilizers: $Z_0 Z_1$ and $Z_1 Z_2$ (check parity of adjacent qubits)
- Syndrome tells us *where* bit-flip errors occurred
- Correction: majority vote on which state it should be

Threshold Theorem: If physical error rate $p < p_{\text{th}}$ (threshold), the logical error rate can be made arbitrarily small by increasing the code distance d:

$$p_L \approx \left(\frac{p}{p_{\text{th}}}\right)^{\lceil d/2 \rceil}$$

For surface codes: $p_{\text{th}} \approx 1\%$ (circuit-level noise).

Task

Simulate the repetition code, observe error correction in action, and verify threshold behavior.

Step-by-Step Instructions

Step 1: Implement the repetition code encoder

```python
import pennylane as qml
from pennylane import numpy as np
import matplotlib.pyplot as plt

def encode_repetition_code(logical_state, wires):
    """Encode a logical qubit into repetition code.

    |0>_L -> |000...0>
    |1>_L -> |111...1>
    """
    # First qubit already has the logical state
    # CNOT from first to all others
    for i in range(1, len(wires)):
        qml.CNOT(wires=[wires[0], wires[i]])
```

Step 2: Apply random bit-flip errors

```python
def apply_bit_flip_errors(wires, error_prob):
    """Apply X errors with probability error_prob."""
    for w in wires:
        # In simulation, we can use a random choice
        if np.random.random() < error_prob:
            qml.PauliX(wires=w)
```

Step 3: Measure syndrome and decode

```python
def measure_syndrome(wires):
    """Measure stabilizers Z_i Z_{i+1}.
    Returns syndrome bits."""
    n = len(wires)
    syndrome = []
    for i in range(n - 1):
        # Syndrome bit = parity of adjacent qubits
        # In real implementation, use ancillas
        syndrome.append(qml.expval(qml.PauliZ(wires[i]) @ qml.PauliZ(wires[i+1])))
    return syndrome

def decode_majority_vote(measurements):
    """Simple majority vote decoder."""
    n_ones = sum(1 for m in measurements if m == 1)
    n_zeros = sum(1 for m in measurements if m == 0)
    return 1 if n_ones > n_zeros else 0
```

Step 4: Simulate logical error rate

```python
def simulate_repetition_code(d, p_error, n_shots=10000):
    """Simulate logical error rate for distance-d repetition code."""
    dev = qml.device("default.qubit", wires=d)

    logical_errors = 0
    for _ in range(n_shots):
        # Encode |0>_L = |00...0>
        # Apply random bit flips
        errors = [1 if np.random.random() < p_error else 0 for _ in range(d)]
        # Count errors
        n_errors = sum(errors)
        # Majority vote: logical error if > d/2 errors
        if n_errors > d // 2:
            logical_errors += 1

    return logical_errors / n_shots

# Sweep error rates for different distances
distances = [3, 5, 7]
error_rates = np.linspace(0.01, 0.5, 20)

plt.figure(figsize=(8, 6))
for d in distances:
    logical_rates = [simulate_repetition_code(d, p) for p in error_rates]
    plt.semilogy(error_rates, logical_rates, '-o', label=f'd={d}')

plt.xlabel('Physical error rate')
plt.ylabel('Logical error rate')
plt.legend()
plt.title('Repetition Code: Threshold Behavior')
plt.axvline(x=0.5, color='r', linestyle='--', label='Threshold p=0.5')
plt.savefig('repetition_code_threshold.png')
```

Step 5: Observe the threshold crossing

All curves should cross at $p \approx 0.5$ (the theoretical threshold). Below threshold, higher distance = lower logical error rate. Above threshold, higher distance makes things worse!

Extension: Surface Code (Optional)

For the surface code, use `stim` and `pymatching`:

```python
# pip install stim pymatching
import stim
import pymatching

# Generate a distance-3 surface code circuit
circuit = stim.Circuit.generated(
    "surface_code:rotated_memory_z",
    distance=3,
    rounds=3,
    after_clifford_depolarization=0.01  # 1% error rate
)

# Sample syndromes and decode
sampler = circuit.compile_detector_sampler()
detector_samples, observable_samples = sampler.sample(
    shots=10000, separate_observables=True
)

# Decode with MWPM
matcher = pymatching.Matching.from_detector_error_model(
    circuit.detector_error_model()
)
predictions = matcher.decode_batch(detector_samples)
logical_errors = np.sum(predictions != observable_samples)
print(f"Logical error rate: {logical_errors / 10000:.4f}")
```

Deliverables

- ✓ Plot: logical error rate vs. physical error rate for $d = 3, 5, 7$ (curves cross at threshold)
- ✓ Verify: below threshold, doubling d approximately squares the error rate
- ✓ Calculate: for $p_L = 10^{-10}$ and $p = 10^{-3}$, what distance is needed?
- ✓ Extension: repeat for surface code and find threshold $\approx 1\%$

Key Takeaways

- **QEC encodes logical qubits into physical qubits** to protect against errors
- **Syndrome measurement** identifies errors without revealing the encoded information
- **Below threshold**, increasing code distance exponentially suppresses errors
- **Surface codes** have $\sim 1\%$ threshold and are the leading candidate for FTQC

F Project 6: Magic State Distillation Analysis — Day 3

Concepts Reinforced: Q37, Q43, Q10 — Magic States, Eastin-Knill, T Gate Cost

Duration: 2 hours **Framework:** NumPy / PennyLane

Why This Matters

The **T gate is the bottleneck** of fault-tolerant quantum computing. You cannot implement T gates directly with QEC — you need **magic state distillation**, which is extremely expensive. Understanding this overhead is crucial for resource estimation and explains why Microsoft's topological approach is compelling.

Theory Background

The Eastin-Knill Theorem: No quantum error correcting code can implement a **universal** gate set transversally (by applying the same gate to each physical qubit). This means at least one gate must be implemented differently.

Clifford Gates vs. Non-Clifford:

- **Clifford gates** (H, S, CNOT): can be implemented transversally or via lattice surgery. "Cheap" in fault-tolerant QC.
- **Non-Clifford gates** (T gate): cannot be transversal. Must use magic states. "Expensive".

Magic States: The state $|T\rangle = \frac{1}{\sqrt{2}}(|0\rangle + e^{i\pi/4}|1\rangle)$ is "magic" because:

1. It cannot be prepared from $|0\rangle$ using only Clifford gates
2. Combined with Clifford gates, it enables universal computation
3. You can "inject" a T gate by teleporting through a magic state

Distillation: Noisy magic states (error rate p) can be purified:

- **15-to-1 protocol**: Take 15 noisy states, get 1 cleaner state with error $p_{\text{out}} = 35p^3$
- Multiple rounds: each round cubes the error (approximately)
- Cost: uses 15^r input states for r rounds of distillation

Why This Is The Bottleneck:

- Algorithms need millions of T gates (e.g., chemistry: $\sim 10^9$ T gates)
- Each T gate consumes one distilled magic state
- Distillation factories occupy many logical qubits
- ~90% of a fault-tolerant computer's qubits may be in T-factories!

Microsoft's Advantage: Topological qubits have inherently lower error rates → need less distillation → ~10× fewer physical qubits.

Task

Model magic state distillation mathematically and quantify the resource overhead.

Step-by-Step Instructions

Step 1: Model the 15-to-1 protocol

```python
import numpy as np
import matplotlib.pyplot as plt

def distill_15_to_1(p_in):
    """15-to-1 magic state distillation.

    Input: error rate p_in
```

```
    Output: error rate p_out = 35 * p_in^3

    This is the leading-order approximation.
    """
    return 35 * p_in**3

def distill_multiple_rounds(p_initial, n_rounds):
    """Apply distillation n_rounds times."""
    p = p_initial
    for _ in range(n_rounds):
        p = distill_15_to_1(p)
    return p
```

Step 2: Analyze error reduction

```
# Starting error rate (typical for encoded magic states)
p_initial = 1e-3

# How many rounds to reach different targets?
targets = [1e-6, 1e-9, 1e-12, 1e-15]

print("Rounds needed to reach target error:")
for target in targets:
    rounds = 0
    p = p_initial
    while p > target and rounds < 10:
        p = distill_15_to_1(p)
        rounds += 1
    print(f"  Target {target:.0e}: {rounds} rounds, final p = {p:.2e}")
```

Step 3: Compute resource cost

```
def magic_state_cost(p_initial, p_target):
    """Compute total magic states consumed to produce one distilled state.
    """
    p = p_initial
    n_rounds = 0
    while p > p_target:
        p = distill_15_to_1(p)
        n_rounds += 1

    # Each round uses 15 input states
    total_input_states = 15 ** n_rounds
    return n_rounds, total_input_states

# Example: Going from p=1e-3 to p=1e-10
rounds, cost = magic_state_cost(1e-3, 1e-10)
print(f"Rounds: {rounds}, Input states per output: {cost}")
```

Step 4: Plot error vs. rounds

```
# Sweep initial error rates
p_initials = [0.1, 0.05, 0.01, 0.005, 0.001]
rounds_range = range(5)

plt.figure(figsize=(10, 6))
for p_init in p_initials:
    errors = [p_init]
    for r in range(1, 5):
        errors.append(distill_multiple_rounds(p_init, r))
```

```
    plt.semilogy(range(len(errors)), errors, 'o-', label=f'p_init={p_init}
        ')

plt.xlabel('Distillation rounds')
plt.ylabel('Magic state error rate')
plt.axhline(y=1e-10, color='r', linestyle='--', label='Typical target')
plt.legend()
plt.title('Magic State Distillation: Error vs Rounds')
plt.grid(True)
plt.savefig('magic_state_distillation.png')
```

Step 5: Estimate physical qubit overhead

```
def estimate_t_factory_qubits(p_physical, p_target, code_distance):
    """Estimate qubits for one T-factory."""
    rounds, states_per_output = magic_state_cost(p_physical, p_target)

    # Each magic state is one logical qubit
    # Each logical qubit needs ~2*d^2 physical qubits (surface code)
    phys_per_logical = 2 * code_distance**2

    # Factory processes 15 states at a time (simplified)
    factory_qubits = 15 * phys_per_logical * rounds

    return factory_qubits, rounds, states_per_output

qubits, rounds, cost = estimate_t_factory_qubits(1e-3, 1e-10, d=17)
print(f"T-factory estimate: {qubits} physical qubits")
print(f"Distillation rounds: {rounds}")
print(f"Magic states consumed per T gate: {cost}")
```

Deliverables

- ✓ Plot: error rate vs. distillation rounds for various initial error rates
- ✓ Table: rounds needed and cost (input states) to reach different target error rates
- ✓ Calculation: T-factory qubit count for a realistic scenario
- ✓ Paragraph: explain why this is the bottleneck and how topological qubits help (lower $p \to$ fewer rounds $\to$ smaller factories)

Key Takeaways

- **T gates require magic states** — cannot be done transversally
- **Distillation is expensive**: 15^r input states for r rounds
- **Lower physical error rate $\to$ fewer distillation rounds $\to$ huge savings**
- This is why **Microsoft's topological qubits** (with inherent protection) are compelling

G Project 7: Azure Quantum Resource Estimation Day 4

Concepts Reinforced: Q66, Q74, Q64 — Resource Estimator, FTQC Cost, MS Stack

Duration: 2 hours **Framework:** Azure Quantum / Q# (browser-based)

Why This Matters

The **Azure Quantum Resource Estimator** is Microsoft's tool for understanding the true cost of fault-tolerant quantum algorithms. Being able to use it and interpret its outputs demonstrates you understand the practical realities of quantum computing — not just the theory.

Theory Background

What Resource Estimation Answers:

- How many **physical qubits** are needed?
- How many **T gates** does the algorithm use?
- How long will the computation take (**runtime**)?
- What **code distance** is required for the target error rate?
- How many **T-factories** run in parallel?

Key Parameters:

- **Error budget**: Total allowed probability of failure (typically 10^{-3} to 10^{-6})
- **Physical error rate**: How noisy are the qubits? (superconducting: $\sim 10^{-3}$, topological target: $\sim 10^{-6}$)
- **QEC scheme**: Surface code, Floquet code, etc.
- **Qubit technology**: Gate-based, Majorana, etc.

Microsoft's Advantage:

- Topological qubits have lower physical error rates $\rightarrow$ smaller code distances
- Floquet codes need $\sim$10$\times$ fewer physical qubits than surface codes
- Combined: potentially 100$\times$ reduction in qubit count

Task

Use the Azure Quantum Resource Estimator to analyze fault-tolerant chemistry costs.

Step-by-Step Instructions

Step 1: Access the Resource Estimator

Go to: https://learn.microsoft.com/en-us/azure/quantum/intro-to-resource-estimation

You can use the browser-based Jupyter notebooks (free Azure account needed) or install locally:

```
pip install azure-quantum qsharp
```

Step 2: Follow the chemistry tutorial

Microsoft provides a detailed tutorial for chemistry resource estimation: https://learn.microsoft.com/en-us/azure/quantum/tutorial-resource-estimator-chemistry

The tutorial walks through estimating resources for a small molecule using the Double-Factorized Qubitization algorithm.

Step 3: Key Q# code structure

```
// Q# code for resource estimation (simplified)
open Microsoft.Quantum.ResourceEstimation;

@EntryPoint()
operation EstimateChemistry() : Unit {
    // The algorithm being estimated
    use qubits = Qubit[numQubits];

    // Apply the chemistry circuit
    // (In practice, use pre-built chemistry libraries)

    // The Resource Estimator counts all operations
}
```

Step 4: Interpret the results

Key outputs to record:

```
# Python code to display results
result = qsharp.estimate("EstimateChemistry()")

print(f"Logical qubits: {result['logicalQubitCount']}")
print(f"Physical qubits: {result['physicalQubitCount']}")
print(f"T gates: {result['tCount']}")
print(f"T factories: {result['numTFactories']}")
print(f"Runtime: {result['runtime']}")
print(f"Code distance: {result['codeDistance']}")
```

Step 5: Compare different configurations

Run the estimator with different parameters:

```
configurations = [
    {"errorBudget": 1e-3, "qubitParams": "surface_code"},
    {"errorBudget": 1e-6, "qubitParams": "surface_code"},
    {"errorBudget": 1e-3, "qubitParams": "floquet_code"},
    {"errorBudget": 1e-3, "qubitParams": "majorana"},
]

results = []
for config in configurations:
    result = qsharp.estimate("EstimateChemistry()", **config)
    results.append({
        "config": config,
        "physical_qubits": result['physicalQubitCount'],
        "runtime": result['runtime'],
        "code_distance": result['codeDistance']
    })

# Create comparison table
import pandas as pd
df = pd.DataFrame(results)
print(df)
```

Deliverables

- ✓ Table: resource estimates for the same algorithm across different configurations
- ✓ Bar chart: physical qubit count for surface code vs. Floquet code vs. Majorana

- ✓ Observation: tighter error budget → larger code distance → more qubits
- ✓ Paragraph: explain how Microsoft's topological + Floquet approach reduces resources

Key Takeaways

- **Resource estimation bridges theory and practice** — shows what's really needed
- **T gates dominate cost** — most physical qubits are in T-factories
- **Error budget choice is critical** — tighter budget → exponentially more qubits
- **Microsoft's approach** (topological + Floquet) can reduce qubits by ∼100×

H Project 8: VQE for LiH with Qubit Reduction Day 5

Concepts Reinforced: Q21, Q28, Q46, Q49, Q50 — Active Space, Qubit Tapering, Symmetries

Duration: 2–3 hours **Framework:** PennyLane

Why This Matters

Real molecules need many more qubits than H_2. LiH needs 12 qubits in minimal basis — already challenging to simulate classically. Learning **qubit reduction techniques** is essential for scaling to larger systems.

Theory Background

The Qubit Scaling Problem:

- Each molecular orbital → 1 qubit (with Jordan-Wigner)
- H_2O in STO-3G: 7 orbitals × 2 spins = 14 qubits
- Industrially relevant molecules: 50–100+ qubits
- Every qubit reduction helps!

Active Space Selection:

- **Frozen core**: Core orbitals (e.g., 1s of Li) don't participate in bonding. Freeze them out.
- **Active space**: Only include orbitals near the HOMO-LUMO gap that contribute to chemistry.
- Notation: CAS(n,m) = n electrons in m orbitals

Qubit Tapering:

Molecular Hamiltonians have symmetries (electron number, spin, point group). These symmetries correspond to $\mathbb{Z}_2$ **operators** that commute with H. We can:

1. Find the symmetry operators (e.g., parity operators)
2. Project onto a specific symmetry sector
3. **Remove qubits** that are fixed by the symmetry

Example: LiH in STO-3G

- Full: 6 orbitals × 2 spins = 12 qubits

- After tapering: typically 8–10 qubits (removing 2–4 qubits via symmetries)
- With active space: could reduce further

Task

Apply qubit reduction techniques to LiH and run VQE on the reduced system.

Step-by-Step Instructions

Step 1: Load LiH and examine the Hamiltonian

```
import pennylane as qml
from pennylane import numpy as np

# Load LiH dataset
dataset = qml.data.load("qchem", molname="LiH",
                        basis="STO-3G", bondlength=1.548)[0]

H = dataset.hamiltonian
hf_state = dataset.hf_state

print(f"Number of qubits (full): {len(hf_state)}")
print(f"Hartree-Fock state: {hf_state}")
print(f"Number of Pauli terms: {len(H.ops)}")
print(f"HF energy: {dataset.hf_energy:.6f} Ha")
```

Step 2: Identify symmetries

PennyLane provides tools to find $\mathbb{Z}_2$ symmetries:

```
# Get molecular parameters
symbols = ["Li", "H"]
geometry = np.array([[0.0, 0.0, 0.0], [0.0, 0.0, 1.548]])

# Build the molecular Hamiltonian
H, n_qubits = qml.qchem.molecular_hamiltonian(
    symbols, geometry, basis="sto-3g"
)

# Find Z2 symmetry generators
generators = qml.qchem.symmetry_generators(H)
print(f"Number of symmetry generators: {len(generators)}")
for i, gen in enumerate(generators):
    print(f"  Generator {i}: {gen}")
```

Step 3: Apply qubit tapering

```
# Get the Pauli-X operators (tapering operators)
paulix_ops = qml.qchem.paulix_ops(generators, n_qubits)

# Optimal sector for the ground state
# (determined by the HF state or trial)
n_electrons = 4  # LiH has 4 electrons

# Taper the Hamiltonian
H_tapered, qubits_tapered = qml.qchem.taper(
    H, generators, paulix_ops, n_electrons
)

```

```python
print(f"Original qubits: {n_qubits}")
print(f"Tapered qubits: {qubits_tapered}")
print(f"Qubits saved: {n_qubits - qubits_tapered}")
```

Step 4: Taper the ansatz operations

```python
# Also taper the HF state and excitations
hf_tapered = qml.qchem.taper_hf(
    generators, paulix_ops, n_electrons,
    num_wires=n_qubits, hf_state=hf_state
)

print(f"Tapered HF state: {hf_tapered}")
```

Step 5: Run VQE on the tapered system

```python
# Device with tapered qubit count
dev = qml.device("default.qubit", wires=qubits_tapered)

# Use AllSinglesDoubles ansatz (adapts to qubit count)
singles, doubles = qml.qchem.excitations(n_electrons, qubits_tapered)
print(f"Single excitations: {len(singles)}")
print(f"Double excitations: {len(doubles)}")

@qml.qnode(dev)
def circuit(params):
    qml.BasisState(hf_tapered, wires=range(qubits_tapered))
    qml.AllSinglesDoubles(params, wires=range(qubits_tapered),
                          hf_state=hf_tapered,
                          singles=singles, doubles=doubles)
    return qml.expval(H_tapered)

# Optimize
n_params = len(singles) + len(doubles)
params = np.zeros(n_params, requires_grad=True)
opt = qml.GradientDescentOptimizer(stepsize=0.4)

energies = []
for step in range(100):
    params, energy = opt.step_and_cost(circuit, params)
    energies.append(energy)
    if step % 20 == 0:
        print(f"Step {step}: E = {energy:.6f} Ha")

# Compare to exact
H_matrix = qml.matrix(H_tapered)
exact_energy = np.min(np.linalg.eigvalsh(H_matrix))
print(f"\nVQE energy:   {energies[-1]:.6f} Ha")
print(f"Exact (FCI):  {exact_energy:.6f} Ha")
print(f"Error: {(energies[-1] - exact_energy)*1000:.2f} mHa")
```

Step 6: Discuss active space (conceptual)

```python
# In a full active space treatment:
# - Li 1s orbital: FROZEN (always doubly occupied)
# - Li 2s, 2p and H 1s: ACTIVE (participate in bonding)
#
# This would reduce from 12 qubits to ~8 qubits
# Combined with tapering: possibly 4-6 qubits

```

```
# The key insight: quantum computers should focus on
# the "hard" part (active space) where classical methods
# struggle, while classical computers handle the easy
# parts (core orbitals, dynamic correlation)
```

Deliverables

- ✓ Table: qubit count at each stage (full → tapered)
- ✓ List: the symmetry generators found and what physical quantity they correspond to
- ✓ VQE energy vs. FCI — is it within chemical accuracy (1.6 mHa)?
- ✓ Discussion: what orbitals would you freeze in an active space approach and why?

Key Takeaways

- **Qubit reduction is essential** for scaling to larger molecules
- **Symmetry tapering** exploits conserved quantities to remove qubits
- **Active space selection** focuses quantum resources on the chemically important orbitals
- **Hybrid classical-quantum**: let classical computers handle the easy parts

I Project 9: Classical Shadows for Efficient Measurement Day 5

Concepts Reinforced: Q26, Q60 — Classical Shadows, Measurement Efficiency

Duration: 2 hours **Framework:** PennyLane + NumPy

Why This Matters

VQE requires measuring many Pauli strings (often hundreds for small molecules). Naively, this requires $O(M \cdot S)$ shots for M observables at precision $\varepsilon \sim 1/\sqrt{S}$. **Classical shadows** reduce this to $O(S)$ total shots for *all* observables — a huge practical speedup.

Theory Background

The Measurement Problem:

- H_2 Hamiltonian has ∼15 Pauli terms
- H_2O might have ∼1000 terms
- To measure each to precision ε, need $\sim 1/\varepsilon^2$ shots each
- Total: $O(M/\varepsilon^2)$ shots — grows linearly with Hamiltonian size

Classical Shadows Idea: Rather than measuring each observable separately, we:

1. Apply a **random unitary** U (from a fixed ensemble) to the state
2. Measure in the computational basis → get outcome $|b\rangle$
3. The "snapshot" is $U^\dagger|b\rangle\langle b|U$
4. **Invert** the measurement channel to get a "classical shadow" $\hat{\rho}$

The Magic: From T shadows, we can estimate *any* observable O:

$$\langle O \rangle \approx \frac{1}{T} \sum_{i=1}^{T} \mathrm{Tr}(\hat{\rho}_i \cdot O)$$

The cost is $O(T)$ regardless of how many observables we want!

Random Pauli Measurements: The simplest shadow protocol:

- For each qubit, randomly choose to measure X, Y, or Z
- Apply the appropriate basis rotation (H for X, H·S† for Y)
- Measure in computational basis

Task

Implement classical shadows from scratch and demonstrate efficient observable estimation.

Step-by-Step Instructions

Step 1: Prepare a test state

```python
import pennylane as qml
from pennylane import numpy as np
import matplotlib.pyplot as plt

# Use the H2 ground state from VQE
n_qubits = 4
dev = qml.device("default.qubit", wires=n_qubits)

# Prepare ground state circuit (from Project 1)
@qml.qnode(dev)
def prepare_state():
    qml.BasisState([1, 1, 0, 0], wires=range(n_qubits))
    qml.DoubleExcitation(0.22, wires=[0, 1, 2, 3])  # Optimized param
    return qml.state()

psi = prepare_state()
```

Step 2: Implement random Pauli measurement

```python
def random_pauli_measurement(state, n_qubits):
    """Perform random Pauli measurement and return snapshot."""
    # Random basis for each qubit: 0=X, 1=Y, 2=Z
    bases = np.random.randint(0, 3, size=n_qubits)

    # Basis rotation unitaries
    # X basis: H
    # Y basis: H @ S^dag = HSdag
    # Z basis: I

    dev = qml.device("default.qubit", wires=n_qubits)

    @qml.qnode(dev)
    def measure_circuit():
        # Load the state
        qml.StatePrep(state, wires=range(n_qubits))

        # Apply basis rotations
        for i, b in enumerate(bases):
```

```
            if b == 0:  # X basis
                qml.Hadamard(wires=i)
            elif b == 1:  # Y basis
                qml.adjoint(qml.S)(wires=i)
                qml.Hadamard(wires=i)
            # Z basis: no rotation needed

        return qml.sample(wires=range(n_qubits))

    outcome = measure_circuit()
    return bases, outcome
```

Step 3: Compute classical shadow from snapshot

```
def snapshot_to_shadow(bases, outcome, n_qubits):
    """Convert measurement snapshot to classical shadow operator."""
    # For random Pauli: shadow = (3|b><b| - I) for each qubit
    # where |b> is the post-measurement state in original basis

    shadow_ops = []
    for i in range(n_qubits):
        b = outcome[i]  # 0 or 1
        basis = bases[i]  # X, Y, or Z

        # Post-measurement state rotated back
        if basis == 0:  # X measurement
            state = np.array([1, (-1)**b]) / np.sqrt(2)  # |+> or |->
        elif basis == 1:  # Y measurement
            state = np.array([1, 1j*(-1)**(b+1)]) / np.sqrt(2)  # |+i> or
                |-i>
        else:  # Z measurement
            state = np.array([1-b, b])  # |0> or |1>

        # Shadow operator: 3|s><s| - I
        shadow_op = 3 * np.outer(state, state.conj()) - np.eye(2)
        shadow_ops.append(shadow_op)

    # Full shadow is tensor product
    full_shadow = shadow_ops[0]
    for op in shadow_ops[1:]:
        full_shadow = np.kron(full_shadow, op)

    return full_shadow
```

Step 4: Estimate observables from shadows

```
def estimate_observable(shadows, observable_matrix):
    """Estimate <O> from classical shadows."""
    estimates = []
    for shadow in shadows:
        # Tr(shadow @ O)
        est = np.real(np.trace(shadow @ observable_matrix))
        estimates.append(est)
    return np.mean(estimates), np.std(estimates) / np.sqrt(len(estimates))

# Collect shadows
n_shadows = 1000
shadows = []
for _ in range(n_shadows):
    bases, outcome = random_pauli_measurement(psi, n_qubits)
```

```
    shadow = snapshot_to_shadow(bases, outcome, n_qubits)
    shadows.append(shadow)
```

Step 5: Compare to exact expectation values

```
# Test with simple Pauli observables
observables = {
    "Z0": np.kron(np.diag([1, -1]), np.eye(8)),
    "Z0Z1": np.kron(np.kron(np.diag([1, -1]), np.diag([1])), np.eye(4)
        ),
}

rho_exact = np.outer(psi, psi.conj())

for name, O in observables.items():
    exact = np.real(np.trace(rho_exact @ O))
    est, err = estimate_observable(shadows, O)
    print(f"{name}: exact={exact:.4f}, shadow={est:.4f} +/- {err:.4f}")
```

Deliverables

- ✓ Working classical shadows implementation
- ✓ Plot: estimation error vs. number of shadows (should scale as $1/\sqrt{T}$)
- ✓ Comparison: shadow estimate vs. exact for several Pauli observables
- ✓ Analysis: total shots for shadows vs. naive measurement of all observables

Key Takeaways

- **Classical shadows reduce measurement cost** from $O(M/\varepsilon^2)$ to $O(\log M/\varepsilon^2)$
- The protocol uses **randomized measurements** with post-processing
- Shadows can estimate *any* linear function of ρ, not just predefined observables
- This is a **practical technique** for near-term quantum computers

J Project 10: Trotter-Suzuki Time Evolution — Day 6

Concepts Reinforced: Q55, Q56, Q80 — Trotterization, Hamiltonian Simulation, Error Bounds

Duration: 2 hours **Framework:** PennyLane

Why This Matters

Simulating time evolution e^{-iHt} is fundamental to both QPE and dynamics simulation. **Trotterization** is the standard technique to decompose e^{-iHt} into implementable gates. Understanding Trotter error is essential for circuit depth estimation.

Theory Background

The Problem: We want to implement $U = e^{-iHt}$ where $H = \sum_k H_k$ (sum of terms). Even if each $e^{-iH_k t}$ is easy, $e^{-i(\sum_k H_k)t} \neq \prod_k e^{-iH_k t}$ because operators don't commute!

First-Order Trotter:

$$e^{-iHt} \approx \left(\prod_k e^{-iH_k t/n} \right)^n + O(t^2/n)$$

Error scales as $O(t^2/n)$ — more steps → smaller error.

Second-Order Trotter (Suzuki):

$$e^{-iHt} \approx \left(\prod_k e^{-iH_k t/2n} \prod_k e^{-iH_k t/2n} \right)^n + O(t^3/n^2)$$

Symmetric ordering cancels first-order error. Error scales as $O(t^3/n^2)$ — quadratically better!

Trade-off: More Trotter steps means:

- Better approximation (smaller Trotter error)
- Deeper circuit (more gates, more hardware noise)

Task

Implement Trotter-based time evolution and verify the error scaling.

Step-by-Step Instructions

Step 1: Define a test Hamiltonian

```python
import pennylane as qml
from pennylane import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import expm

# Heisenberg model on 4 qubits: H = J*sum(XX + YY + ZZ) + h*sum(Z)
n_qubits = 4
J = 1.0
h = 0.5

def build_hamiltonian_matrix():
    """Build the full Hamiltonian matrix."""
    dim = 2**n_qubits
    H = np.zeros((dim, dim), dtype=complex)

    # Pauli matrices
    I = np.eye(2)
    X = np.array([[0, 1], [1, 0]])
    Y = np.array([[0, -1j], [1j, 0]])
    Z = np.array([[1, 0], [0, -1]])

    for i in range(n_qubits - 1):
        # XX term
        ops = [I]*n_qubits
        ops[i] = X
        ops[i+1] = X
        term = ops[0]
        for op in ops[1:]:
            term = np.kron(term, op)
        H += J * term

        # YY and ZZ terms (similar)
        # ... (add YY and ZZ contributions)

```

```
    # Field term
    for i in range(n_qubits):
        ops = [I]*n_qubits
        ops[i] = Z
        term = ops[0]
        for op in ops[1:]:
            term = np.kron(term, op)
        H += h * term

    return H

H_matrix = build_hamiltonian_matrix()
```

Step 2: Exact time evolution

```
def exact_evolution(H, t, psi_0):
    """Exact time evolution via matrix exponentiation."""
    U = expm(-1j * H * t)
    return U @ psi_0

# Initial state |0110>
psi_0 = np.zeros(2**n_qubits)
psi_0[6] = 1.0  # |0110> = binary 6

t = 1.0
psi_exact = exact_evolution(H_matrix, t, psi_0)
```

Step 3: First-order Trotter circuit

```
dev = qml.device("default.qubit", wires=n_qubits)

def trotter_step_first_order(t, wires):
    """One first-order Trotter step."""
    n = len(wires)
    dt = t

    # XX + YY + ZZ interactions
    for i in range(n - 1):
        qml.IsingXX(2*J*dt, wires=[wires[i], wires[i+1]])
        qml.IsingYY(2*J*dt, wires=[wires[i], wires[i+1]])
        qml.IsingZZ(2*J*dt, wires=[wires[i], wires[i+1]])

    # Field terms
    for i in range(n):
        qml.RZ(2*h*dt, wires=wires[i])

@qml.qnode(dev)
def trotter_circuit(n_steps, t_total):
    # Initial state |0110>
    qml.PauliX(wires=1)
    qml.PauliX(wires=2)

    # Trotter steps
    dt = t_total / n_steps
    for _ in range(n_steps):
        trotter_step_first_order(dt, range(n_qubits))

    return qml.state()
```

Step 4: Second-order Trotter circuit

```python
def trotter_step_second_order(t, wires):
    """One second-order (symmetric) Trotter step."""
    n = len(wires)
    dt = t / 2  # Half step

    # Forward direction
    for i in range(n - 1):
        qml.IsingXX(2*J*dt, wires=[wires[i], wires[i+1]])
        qml.IsingYY(2*J*dt, wires=[wires[i], wires[i+1]])
        qml.IsingZZ(2*J*dt, wires=[wires[i], wires[i+1]])
    for i in range(n):
        qml.RZ(2*h*dt, wires=wires[i])

    # Backward direction (same operations, achieves symmetric ordering)
    for i in range(n):
        qml.RZ(2*h*dt, wires=wires[i])
    for i in range(n-2, -1, -1):
        qml.IsingZZ(2*J*dt, wires=[wires[i], wires[i+1]])
        qml.IsingYY(2*J*dt, wires=[wires[i], wires[i+1]])
        qml.IsingXX(2*J*dt, wires=[wires[i], wires[i+1]])
```

Step 5: Measure fidelity vs. Trotter steps

```python
def fidelity(psi1, psi2):
    """State fidelity |<psi1|psi2>|^2."""
    return np.abs(np.vdot(psi1, psi2))**2

trotter_steps = [1, 2, 4, 8, 16, 32, 64]
errors_first = []
errors_second = []

for n in trotter_steps:
    # First-order
    psi_trotter = trotter_circuit(n, t)
    f = fidelity(psi_exact, psi_trotter)
    errors_first.append(1 - f)

    # Second-order (similar implementation)
    # ...

# Plot
plt.figure(figsize=(8, 6))
plt.loglog(trotter_steps, errors_first, 'o-', label='First-order')
plt.loglog(trotter_steps, errors_second, 's-', label='Second-order')
plt.xlabel('Trotter steps')
plt.ylabel('Infidelity (1 - F)')
plt.legend()
plt.title('Trotter Error Scaling')
plt.savefig('trotter_error.png')
```

Deliverables

- ✓ Log-log plot: infidelity vs. Trotter steps for first- and second-order
- ✓ Fitted slopes: first-order $\sim 1/n$, second-order $\sim 1/n^2$
- ✓ Table: circuit depth (gate count) vs. Trotter steps
- ✓ Analysis: at what n does Trotter error match hardware noise?

Key Takeaways

- **Trotterization** is the standard way to implement e^{-iHt} on quantum hardware
- **Second-order Trotter** is much more accurate for the same depth
- There's a **depth-accuracy trade-off** — more steps help until hardware noise dominates
- For fault-tolerant QC, Trotter steps can be arbitrarily deep; for NISQ, we're limited

K Project 11: Noise Simulation & Zero-Noise Extrapolation Day 6

Concepts Reinforced: Q59, Q8, Q84, Q85 — Error Mitigation, Decoherence, Noise Models

Duration: 2 hours **Framework:** PennyLane (`default.mixed`)

Why This Matters

NISQ devices have significant noise. **Error mitigation** techniques extract better results from noisy hardware without full QEC. **Zero-Noise Extrapolation (ZNE)** is one of the most practical and widely-used techniques.

Theory Background

Types of Noise:

- **Depolarizing**: Random Pauli error with probability p. Models general decoherence.

$$\mathcal{E}(\rho) = (1-p)\rho + \frac{p}{3}(X\rho X + Y\rho Y + Z\rho Z)$$

- **Amplitude damping**: Models energy relaxation (T_1 decay). $|1\rangle \to |0\rangle$ with some probability.
- **Dephasing**: Models phase randomization (T_2 decay). $|+\rangle \to$ mixed state.

Zero-Noise Extrapolation (ZNE):

Key insight: If we could run at different noise levels, we could extrapolate to zero noise!

How to amplify noise:

- **Pulse stretching**: Slow down pulses $\to$ more time for decoherence
- **Unitary folding**: Replace U with $U \cdot U^\dagger \cdot U$ (same effect, $3\times$ errors)

ZNE Procedure:

1. Run circuit at noise scale factors $\lambda = 1, 2, 3, \ldots$
2. Fit expectation value $E(\lambda)$ to a model (linear, polynomial, exponential)
3. Extrapolate to $\lambda = 0$ to get the noise-free estimate

Task

Simulate VQE under noise and implement ZNE to recover the correct energy.

Step-by-Step Instructions

Step 1: Setup noisy simulation

```python
import pennylane as qml
from pennylane import numpy as np
import matplotlib.pyplot as plt

# Load H2 data
dataset = qml.data.load("qchem", molname="H2",
                        basis="STO-3G", bondlength=0.742)[0]
H = dataset.hamiltonian
hf_state = [1, 1, 0, 0]
n_qubits = 4

# Use mixed-state simulator for noise
dev_noisy = qml.device("default.mixed", wires=n_qubits)

# Optimal VQE parameter (from noiseless optimization)
theta_opt = 0.22
```

Step 2: Create noisy VQE circuit

```python
def create_noisy_circuit(noise_prob):
    """Create VQE circuit with depolarizing noise after each gate."""

    @qml.qnode(dev_noisy)
    def circuit():
        # Prepare HF state (with noise)
        qml.BasisState(hf_state, wires=range(n_qubits))

        # Add noise after state prep
        for i in range(n_qubits):
            qml.DepolarizingChannel(noise_prob, wires=i)

        # Double excitation (decomposed into native gates)
        # Each gate gets noise
        qml.DoubleExcitation(theta_opt, wires=[0, 1, 2, 3])

        # Add noise after the ansatz
        for i in range(n_qubits):
            qml.DepolarizingChannel(noise_prob, wires=i)

        return qml.expval(H)

    return circuit

# Exact (noiseless) energy
dev_exact = qml.device("default.qubit", wires=n_qubits)

@qml.qnode(dev_exact)
def exact_circuit():
    qml.BasisState(hf_state, wires=range(n_qubits))
    qml.DoubleExcitation(theta_opt, wires=[0, 1, 2, 3])
    return qml.expval(H)

E_exact = exact_circuit()
print(f"Exact VQE energy: {E_exact:.6f} Ha")
```

Step 3: Implement unitary folding for noise amplification

```python
def create_folded_circuit(noise_prob, scale_factor):
    """Circuit with noise amplified by scale_factor via folding."""

    @qml.qnode(dev_noisy)
    def circuit():
        # Prepare HF state
        qml.BasisState(hf_state, wires=range(n_qubits))
        for i in range(n_qubits):
            qml.DepolarizingChannel(noise_prob, wires=i)

        # Original ansatz
        qml.DoubleExcitation(theta_opt, wires=[0, 1, 2, 3])
        for i in range(n_qubits):
            qml.DepolarizingChannel(noise_prob, wires=i)

        # Folding: apply U^dag U for (scale_factor-1) times
        for _ in range(scale_factor - 1):
            # U^dagger
            qml.adjoint(qml.DoubleExcitation)(theta_opt, wires=[0, 1, 2,
                3])
            for i in range(n_qubits):
                qml.DepolarizingChannel(noise_prob, wires=i)
            # U again
            qml.DoubleExcitation(theta_opt, wires=[0, 1, 2, 3])
            for i in range(n_qubits):
                qml.DepolarizingChannel(noise_prob, wires=i)

        return qml.expval(H)

    return circuit
```

Step 4: Run ZNE

```python
def zne_extrapolation(noise_prob, scale_factors=[1, 2, 3]):
    """Perform zero-noise extrapolation."""

    energies = []
    for s in scale_factors:
        circuit = create_folded_circuit(noise_prob, s)
        E = circuit()
        energies.append(E)
        print(f"  Scale {s}: E = {E:.6f} Ha")

    # Linear extrapolation
    coeffs = np.polyfit(scale_factors, energies, 1)
    E_zne_linear = coeffs[1]  # Value at lambda=0

    # Quadratic extrapolation (if enough points)
    if len(scale_factors) >= 3:
        coeffs_quad = np.polyfit(scale_factors, energies, 2)
        E_zne_quad = coeffs_quad[2]
    else:
        E_zne_quad = None

    return energies, E_zne_linear, E_zne_quad

# Test at different noise levels
noise_levels = [0.001, 0.005, 0.01, 0.02]

```

```python
results = []
for p in noise_levels:
    print(f"\nNoise probability = {p}")
    raw_energy = create_noisy_circuit(p)()
    energies, E_linear, E_quad = zne_extrapolation(p)

    results.append({
        'noise': p,
        'raw': raw_energy,
        'zne_linear': E_linear,
        'zne_quad': E_quad,
        'exact': E_exact
    })

    print(f"  Raw energy:    {raw_energy:.6f} Ha (error: {(raw_energy-E_exact)*1000:.2f} mHa)")
    print(f"  ZNE (linear):  {E_linear:.6f} Ha (error: {(E_linear-E_exact)*1000:.2f} mHa)")
```

Step 5: Visualize results

```python
fig, ax = plt.subplots(figsize=(8, 6))

noise = [r['noise'] for r in results]
raw = [r['raw'] for r in results]
zne = [r['zne_linear'] for r in results]

ax.axhline(y=E_exact, color='k', linestyle='--', label='Exact')
ax.plot(noise, raw, 'ro-', label='Raw (noisy)')
ax.plot(noise, zne, 'bs-', label='ZNE (linear)')

ax.set_xlabel('Noise probability')
ax.set_ylabel('Energy (Ha)')
ax.legend()
ax.set_title('Zero-Noise Extrapolation for VQE')
plt.savefig('zne_results.png')
```

Deliverables

- ✓ Plot: energy vs. noise scale factor showing linear/quadratic fits
- ✓ Table: raw energy, ZNE energy, and exact energy at each noise level
- ✓ Analysis: at what noise level does ZNE fail to achieve chemical accuracy?
- ✓ Observation: ZNE reduces error but doesn't eliminate it completely

Key Takeaways

- **Error mitigation** is essential for useful NISQ results
- **ZNE** extrapolates from amplified noise to estimate zero-noise result
- ZNE has **limited range** — fails when noise is too high
- **Combining techniques** (ZNE + measurement mitigation + other methods) works best

L Project 12: ADAPT-VQE for Efficient Ansatz Construction Day 6

Concepts Reinforced: Q24, Q22, Q25 — ADAPT-VQE, Operator Pools, Gradient Screening

Duration: 2–3 hours **Framework:** PennyLane

Why This Matters

Fixed ansätze like UCCSD may have too many parameters (wasteful) or too few (can't reach ground state). **ADAPT-VQE** builds the ansatz adaptively, adding only the operators that matter. This is a major research direction for practical quantum chemistry.

Theory Background

The Problem with Fixed Ansätze:

- **UCCSD**: Includes all single and double excitations. For large molecules, this is thousands of parameters, but many contribute negligibly.
- **Hardware-efficient**: No chemical motivation. May miss important operators or include unnecessary ones.

ADAPT-VQE Idea: Start with just the HF state, then iteratively:

1. Compute $\nabla_\theta E$ for each operator in the pool (at $\theta = 0$)
2. Select the operator with the **largest gradient** — this contributes most to energy lowering
3. Add it to the ansatz and re-optimize all parameters
4. Repeat until gradients are small (converged)

Operator Pool: All single and double fermionic excitations:

$$\tau_{ij}^{ab} = a_a^\dagger a_i - a_i^\dagger a_a \quad \text{(singles)}$$

$$\tau_{ijkl}^{abcd} = a_a^\dagger a_b^\dagger a_i a_j - a_j^\dagger a_i^\dagger a_b a_a \quad \text{(doubles)}$$

Benefits:

- **Compact ansatz**: Only necessary operators are included
- **Avoids barren plateaus**: Structured growth from HF state
- **Systematic improvement**: Can trade off accuracy vs. circuit depth

Task

Implement ADAPT-VQE and compare to fixed-ansatz VQE.

Step-by-Step Instructions

Step 1: Setup and operator pool

```
import pennylane as qml
from pennylane import numpy as np

# Load LiH data (more interesting than H2)
```

```python
dataset = qml.data.load("qchem", molname="LiH",
                        basis="STO-3G", bondlength=1.548)[0]
H = dataset.hamiltonian
hf_state = dataset.hf_state
n_qubits = len(hf_state)
n_electrons = sum(hf_state)

# Generate operator pool
singles, doubles = qml.qchem.excitations(n_electrons, n_qubits)
print(f"Pool size: {len(singles)} singles + {len(doubles)} doubles")

# Create the operator pool
def create_operator_pool():
    """Create list of excitation operators."""
    pool = []

    # Single excitations
    for s in singles:
        pool.append(('single', s))

    # Double excitations
    for d in doubles:
        pool.append(('double', d))

    return pool

operator_pool = create_operator_pool()
```

Step 2: Compute gradients for operator selection

```python
dev = qml.device("default.qubit", wires=n_qubits)

def compute_operator_gradient(ansatz_ops, ansatz_params, candidate_op):
    """Compute gradient of energy w.r.t. candidate operator at theta=0."""

    @qml.qnode(dev)
    def circuit(params):
        # HF state
        qml.BasisState(hf_state, wires=range(n_qubits))

        # Current ansatz
        for i, (op_type, wires) in enumerate(ansatz_ops):
            if op_type == 'single':
                qml.SingleExcitation(params[i], wires=wires)
            else:
                qml.DoubleExcitation(params[i], wires=wires)

        # Candidate operator at theta=0 (no effect yet)
        # We compute gradient via parameter-shift
        op_type, wires = candidate_op
        if op_type == 'single':
            qml.SingleExcitation(params[-1], wires=wires)
        else:
            qml.DoubleExcitation(params[-1], wires=wires)

        return qml.expval(H)

    # Parameter-shift for gradient at theta=0
    params = list(ansatz_params) + [0.0]
    params = np.array(params, requires_grad=True)
```

```python

    # Gradient of last parameter (candidate)
    E_plus = circuit(params.at[-1].set(np.pi/2))
    E_minus = circuit(params.at[-1].set(-np.pi/2))
    grad = (E_plus - E_minus) / 2

    return np.abs(grad)
```

Step 3: ADAPT-VQE main loop

```python
def adapt_vqe(max_iterations=10, grad_threshold=1e-3):
    """Run ADAPT-VQE algorithm."""

    ansatz_ops = []
    ansatz_params = np.array([], requires_grad=True)
    available_ops = operator_pool.copy()

    history = []

    for iteration in range(max_iterations):
        print(f"\n=== ADAPT Iteration {iteration + 1} ===")

        # Compute gradients for all unused operators
        gradients = []
        for op in available_ops:
            grad = compute_operator_gradient(ansatz_ops, ansatz_params, op
                )
            gradients.append(grad)

        # Find best operator
        max_grad_idx = np.argmax(gradients)
        max_grad = gradients[max_grad_idx]
        print(f"Max gradient: {max_grad:.6f}")

        # Check convergence
        if max_grad < grad_threshold:
            print("Converged!")
            break

        # Add best operator to ansatz
        best_op = available_ops.pop(max_grad_idx)
        ansatz_ops.append(best_op)
        ansatz_params = np.append(ansatz_params, 0.0)
        ansatz_params = np.array(ansatz_params, requires_grad=True)

        print(f"Added operator: {best_op}")

        # Re-optimize all parameters
        @qml.qnode(dev)
        def full_circuit(params):
            qml.BasisState(hf_state, wires=range(n_qubits))
            for i, (op_type, wires) in enumerate(ansatz_ops):
                if op_type == 'single':
                    qml.SingleExcitation(params[i], wires=wires)
                else:
                    qml.DoubleExcitation(params[i], wires=wires)
            return qml.expval(H)

        opt = qml.GradientDescentOptimizer(stepsize=0.4)
        for _ in range(50):
```

```
            ansatz_params = opt.step(full_circuit, ansatz_params)

        energy = full_circuit(ansatz_params)
        print(f"Energy: {energy:.6f} Ha")
        history.append({'iteration': iteration, 'energy': energy,
                        'n_params': len(ansatz_params)})

    return ansatz_ops, ansatz_params, history

# Run ADAPT-VQE
final_ops, final_params, adapt_history = adapt_vqe()
```

Step 4: Compare to fixed UCCSD

```
# Fixed UCCSD for comparison
@qml.qnode(dev)
def uccsd_circuit(params):
    qml.BasisState(hf_state, wires=range(n_qubits))
    qml.AllSinglesDoubles(params, wires=range(n_qubits),
                          hf_state=hf_state,
                          singles=singles, doubles=doubles)
    return qml.expval(H)

n_uccsd_params = len(singles) + len(doubles)
uccsd_params = np.zeros(n_uccsd_params, requires_grad=True)

opt = qml.GradientDescentOptimizer(stepsize=0.4)
for _ in range(100):
    uccsd_params = opt.step(uccsd_circuit, uccsd_params)

E_uccsd = uccsd_circuit(uccsd_params)
E_adapt = adapt_history[-1]['energy']

print(f"\nComparison:")
print(f"UCCSD:  E = {E_uccsd:.6f} Ha, params = {n_uccsd_params}")
print(f"ADAPT:  E = {E_adapt:.6f} Ha, params = {len(final_params)}")
```

Deliverables

- ✓ Plot: energy vs. ADAPT iteration
- ✓ Table: ADAPT vs. UCCSD — # parameters, energy, circuit depth
- ✓ Bar plot: gradient magnitudes at first iteration (shows which operators matter)
- ✓ Observation: ADAPT achieves similar accuracy with fewer parameters

Key Takeaways

- **ADAPT-VQE builds compact ansätze** tailored to each molecule
- **Gradient screening** identifies chemically important operators
- Typical savings: 50–90% fewer parameters than fixed UCCSD
- **Trade-off**: More measurement overhead in gradient computation

M Project 13: Quantum Teleportation & Gate Teleportation Day 7

Concepts Reinforced: Q12, Q37, Q82 — Teleportation, Gate Teleportation, FTQC Mechanism

Duration: 1.5 hours **Framework:** PennyLane

Why This Matters

Gate teleportation is how fault-tolerant quantum computers implement non-Clifford gates (like T). Understanding this mechanism explains why magic state distillation is central to FTQC and connects to measurement-based quantum computing concepts.

Theory Background

Standard Quantum Teleportation: Transfers an unknown state $|\psi\rangle$ from Alice to Bob using:

- 1 shared Bell pair (pre-distributed entanglement)
- 2 classical bits (measurement outcomes)
- Local operations (Pauli corrections)

Protocol:

1. Alice has $|\psi\rangle$ and one half of Bell pair $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$
2. Alice performs Bell measurement on her two qubits
3. Alice sends 2-bit outcome to Bob
4. Bob applies Pauli correction: $Z^{m_1}X^{m_2}$ where m_1, m_2 are measurement results
5. Bob now has $|\psi\rangle$!

Gate Teleportation: Instead of teleporting $|\psi\rangle$, teleport $G|\psi\rangle$ where G is the desired gate!

T-Gate Teleportation:

1. Prepare a "magic state" $|T\rangle = \frac{1}{\sqrt{2}}(|0\rangle + e^{i\pi/4}|1\rangle)$
2. Teleport data qubit through the magic state
3. Apply S correction if measurement outcome is 1
4. Result: $T|\psi\rangle$ on the output!

Why This Works: The magic state "contains" the T gate. The correction $S = T^2$ commutes through Cliffords, so error correction can handle it.

Task

Implement teleportation and gate teleportation circuits.

Step-by-Step Instructions

Step 1: Standard teleportation

```
import pennylane as qml
from pennylane import numpy as np

```

```python
dev = qml.device("default.qubit", wires=3)

@qml.qnode(dev)
def teleportation_circuit(input_state_params):
    """Teleport a state from qubit 0 to qubit 2."""
    theta, phi = input_state_params

    # Prepare input state on qubit 0
    # |psi> = cos(theta/2)|0> + e^{i*phi}*sin(theta/2)|1>
    qml.RY(theta, wires=0)
    qml.RZ(phi, wires=0)

    # Create Bell pair between qubits 1 and 2
    qml.Hadamard(wires=1)
    qml.CNOT(wires=[1, 2])

    # Bell measurement on qubits 0 and 1
    qml.CNOT(wires=[0, 1])
    qml.Hadamard(wires=0)

    # Measure qubits 0 and 1 (classically controlled)
    # In PennyLane, we use mid-circuit measurements + cond
    m0 = qml.measure(0)
    m1 = qml.measure(1)

    # Apply corrections to qubit 2
    qml.cond(m1, qml.PauliX)(wires=2)
    qml.cond(m0, qml.PauliZ)(wires=2)

    # Return Bloch vector of output qubit
    return [qml.expval(qml.PauliX(2)),
            qml.expval(qml.PauliY(2)),
            qml.expval(qml.PauliZ(2))]

# Test with random input states
def test_teleportation(n_tests=10):
    print("Testing quantum teleportation...")
    for i in range(n_tests):
        theta = np.random.uniform(0, np.pi)
        phi = np.random.uniform(0, 2*np.pi)

        # Expected Bloch vector of input state
        expected_x = np.sin(theta) * np.cos(phi)
        expected_y = np.sin(theta) * np.sin(phi)
        expected_z = np.cos(theta)

        # Run teleportation
        output = teleportation_circuit([theta, phi])

        # Compare (allow for some variance due to mid-circuit measurement)
        print(f"  Test {i+1}: Expected Z={expected_z:.3f}, Got Z={output
            [2]:.3f}")

test_teleportation()
```

Step 2: Gate teleportation for T gate

```python
@qml.qnode(dev)
def t_gate_teleportation(input_state_params):
    """Apply T gate via gate teleportation."""
```

```
    theta, phi = input_state_params

    # Prepare input state on qubit 0
    qml.RY(theta, wires=0)
    qml.RZ(phi, wires=0)

    # Prepare magic state on qubit 1
    # |T> = (|0> + e^{i*pi/4}|1>)/sqrt(2)
    qml.Hadamard(wires=1)
    qml.T(wires=1)

    # Gate teleportation circuit
    qml.CNOT(wires=[0, 1])
    qml.Hadamard(wires=0)

    # Measure qubit 0
    m = qml.measure(0)

    # Apply S correction if m=1
    qml.cond(m, qml.S)(wires=1)

    # Output is on qubit 1 = T|psi>
    return qml.state()

@qml.qnode(dev)
def direct_t_gate(input_state_params):
    """Direct T gate application for comparison."""
    theta, phi = input_state_params

    qml.RY(theta, wires=0)
    qml.RZ(phi, wires=0)
    qml.T(wires=0)

    return qml.state()

# Compare gate teleportation to direct T
def test_gate_teleportation(n_tests=5):
    print("\nTesting T-gate teleportation...")

    for i in range(n_tests):
        theta = np.random.uniform(0, np.pi)
        phi = np.random.uniform(0, 2*np.pi)

        # Direct T gate
        psi_direct = direct_t_gate([theta, phi])

        # Gate teleportation (multiple runs due to measurement)
        # Compare fidelity
        print(f"  Test {i+1}: theta={theta:.3f}, phi={phi:.3f}")

test_gate_teleportation()
```

Step 3: Explain the connection to FTQC

```
"""
CONNECTION TO FAULT-TOLERANT QC:

1. Clifford gates (H, S, CNOT) can be done transversally on surface codes
   -> They are "cheap" in terms of overhead

```

```
2. T gate cannot be done transversally (Eastin-Knill theorem)
   -> Must use gate teleportation

3. Magic states must be prepared at high fidelity
   -> Distillation uses many noisy magic states to make one clean one

4. Each T gate in your algorithm:
   - Consumes one distilled magic state
   - Requires a distillation factory
   - The factory is the dominant qubit cost!

5. Why Microsoft's approach helps:
   - Topological qubits have lower base error rates
   - Fewer distillation rounds needed
   - Smaller T-factories
   - -> Fewer total physical qubits
"""
```

Deliverables

- ✓ Working teleportation circuit with mid-circuit measurements
- ✓ Verified teleportation fidelity > 0.99 for random input states
- ✓ T-gate teleportation circuit that matches direct T application
- ✓ Written explanation connecting gate teleportation to magic state distillation

Key Takeaways

- **Teleportation** uses entanglement + measurement to transfer states
- **Gate teleportation** applies gates by teleporting through a special state
- **T gates via magic states** is the standard FTQC method for non-Cliffords
- This explains why **T-count** is a key metric for quantum algorithms

N Project 14: Grover's Search Algorithm Day 7

Concepts Reinforced: Q79, Q76 — Grover's Search, Universal Gates, Oracle Construction

Duration: 1.5 hours **Framework:** PennyLane

Why This Matters

Grover's algorithm demonstrates **quantum speedup** — $O(\sqrt{N})$ vs. $O(N)$ classically. It's a foundational algorithm that shows how quantum interference can accelerate search problems, and the oracle construction technique applies to many quantum algorithms.

Theory Background

The Problem: Given a function $f : \{0,1\}^n \to \{0,1\}$ that outputs 1 for exactly one input (the "marked" item), find that input.

Classical: Must check inputs one-by-one. Expected $N/2 = 2^{n-1}$ queries.

Quantum (Grover): Only $O(\sqrt{N}) = O(\sqrt{2^n})$ queries!

The Algorithm:

1. Start in uniform superposition: $|s\rangle = H^{\otimes n}|0\rangle = \frac{1}{\sqrt{N}}\sum_x |x\rangle$
2. Repeat $\approx \frac{\pi}{4}\sqrt{N}$ times:
   (a) **Oracle**: Flip phase of marked state: $|x\rangle \to (-1)^{f(x)}|x\rangle$
   (b) **Diffusion**: Reflect about $|s\rangle$: $2|s\rangle\langle s| - I$
3. Measure — high probability of getting marked item!

Geometric Picture:

- State lives in 2D plane: marked $|w\rangle$ and unmarked $|s'\rangle$
- Oracle reflects about $|s'\rangle$ axis
- Diffusion reflects about $|s\rangle$ axis
- Net effect: rotation toward $|w\rangle$ by angle 2θ where $\sin\theta = 1/\sqrt{N}$

Oracle Construction: For a marked state $|w\rangle$, the oracle is:

$$O_w = I - 2|w\rangle\langle w|$$

Implemented using multi-controlled Z gates with X gates to flip the target pattern.

Task

Implement Grover's algorithm from scratch and verify the quadratic speedup.

Step-by-Step Instructions

Step 1: Implement the oracle

```python
import pennylane as qml
from pennylane import numpy as np
import matplotlib.pyplot as plt

def grover_oracle(marked_state, wires):
    """Oracle that flips phase of marked state.

    marked_state: binary list, e.g., [1, 0, 1] for |101>
    """
    n = len(wires)

    # Flip qubits where marked_state is 0
    for i, bit in enumerate(marked_state):
        if bit == 0:
            qml.PauliX(wires=wires[i])

    # Multi-controlled Z (flip phase when all qubits are |1>)
    qml.ctrl(qml.PauliZ, control=wires[:-1])(wires=wires[-1])

    # Unflip qubits
    for i, bit in enumerate(marked_state):
        if bit == 0:
            qml.PauliX(wires=wires[i])
```

Step 2: Implement the diffusion operator

```python
def diffusion_operator(wires):
    """Reflect about the uniform superposition state.

    Implements 2|s><s| - I where |s> = H^n|0>
    """
    n = len(wires)

    # Apply H to go to computational basis
    for w in wires:
        qml.Hadamard(wires=w)

    # Apply X to flip (so we're reflecting about |0...0>)
    for w in wires:
        qml.PauliX(wires=w)

    # Multi-controlled Z
    qml.ctrl(qml.PauliZ, control=wires[:-1])(wires=wires[-1])

    # Undo X and H
    for w in wires:
        qml.PauliX(wires=w)
    for w in wires:
        qml.Hadamard(wires=w)
```

Step 3: Full Grover circuit

```python
def optimal_iterations(n_qubits):
    """Optimal number of Grover iterations."""
    N = 2**n_qubits
    return int(np.round(np.pi/4 * np.sqrt(N)))

def grover_circuit(n_qubits, marked_state, n_iterations):
    """Build Grover's algorithm circuit."""
    dev = qml.device("default.qubit", wires=n_qubits)

    @qml.qnode(dev)
    def circuit():
        wires = range(n_qubits)

        # Initial superposition
        for w in wires:
            qml.Hadamard(wires=w)

        # Grover iterations
        for _ in range(n_iterations):
            grover_oracle(marked_state, wires)
            diffusion_operator(wires)

        # Return probabilities
        return qml.probs(wires=wires)

    return circuit

# Test with n=4 qubits, mark state |7> = |0111>
n = 4
marked = [0, 1, 1, 1]  # Binary for 7
opt_iter = optimal_iterations(n)

circuit = grover_circuit(n, marked, opt_iter)
```

```python
probs = circuit()

print(f"n = {n} qubits, N = {2**n} states")
print(f"Optimal iterations: {opt_iter}")
print(f"Probability of marked state (7): {probs[7]:.4f}")
print(f"Expected: ~{1.0:.4f}")
```

Step 4: Observe Grover oscillation

```python
# Plot probability of success vs. number of iterations
n = 4
marked = [0, 1, 1, 1]
marked_idx = 7

iterations_range = range(0, 15)
success_probs = []

for n_iter in iterations_range:
    circuit = grover_circuit(n, marked, n_iter)
    probs = circuit()
    success_probs.append(probs[marked_idx])

plt.figure(figsize=(8, 5))
plt.plot(iterations_range, success_probs, 'o-')
plt.axhline(y=1.0, color='r', linestyle='--', alpha=0.5)
plt.xlabel('Number of Grover iterations')
plt.ylabel('Probability of finding marked state')
plt.title(f'Grover Oscillation (n={n} qubits)')
plt.grid(True)
plt.savefig('grover_oscillation.png')
```

Step 5: Verify quadratic speedup

```python
# Optimal iterations vs qubit count
qubit_counts = [2, 3, 4, 5, 6]
iterations_needed = []
classical_queries = []

for n in qubit_counts:
    N = 2**n
    opt = optimal_iterations(n)
    iterations_needed.append(opt)
    classical_queries.append(N // 2)  # Expected classical

plt.figure(figsize=(8, 5))
plt.semilogy(qubit_counts, classical_queries, 's-', label='Classical O(N)')
plt.semilogy(qubit_counts, iterations_needed, 'o-', label='Grover O(sqrt(N))')
plt.xlabel('Number of qubits (n)')
plt.ylabel('Queries needed')
plt.legend()
plt.title('Grover\'s Quadratic Speedup')
plt.grid(True)
plt.savefig('grover_speedup.png')
```

Deliverables

- ✓ Working Grover implementation with oracle + diffusion

- ✓ Plot: Grover oscillation showing probability vs. iterations
- ✓ Plot: optimal iterations vs. n confirming $O(\sqrt{2^n})$ scaling
- ✓ Table: success probability at optimal iteration for $n = 3, 4, 5, 6$

Key Takeaways

- **Grover's algorithm** achieves quadratic speedup for unstructured search
- The speedup comes from **quantum interference** — amplitude amplification
- **Too many iterations** is bad — probability oscillates, don't overshoot!
- **Oracle construction** is the hard part for real applications

Project Summary & Day Mapping

#	Project	Day	Hrs	Key Topics
1	VQE for H_2 — Full Pipeline	1	2–3	VQE, JW transform, param-shift
2	Potential Energy Surface	1	2	Chemical accuracy, static correlation
3	Ansätze & Barren Plateaus	2	2–3	Ansatz design, gradient variance
4	QPE from Scratch	2	2–3	QPE, QFT, Trotterization
5	QEC Simulation	3	2–3	Surface code, threshold, decoding
6	Magic State Distillation	3	2	T gate cost, Eastin-Knill
7	Azure Resource Estimation	4	2	MS stack, FTQC overhead
8	VQE for LiH (Tapering)	5	2–3	Active space, qubit reduction
9	Classical Shadows	5	2	Measurement efficiency
10	Trotter-Suzuki Simulation	6	2	Error scaling, depth–accuracy
11	Noise & ZNE Mitigation	6	2	Error mitigation, noise models
12	ADAPT-VQE	6	2–3	Adaptive ansatz, operator pools
13	Gate Teleportation	7	1.5	FTQC mechanism, magic states
14	Grover's Algorithm	7	1.5	Oracle design, quadratic speedup
	Total		**~30**	

Table 6: Coding projects mapped to the 7-day study plan. Each includes theory, code, and deliverables.

Prioritization

If time is limited, focus on projects marked with **Day 1, 3, 4, 5** — these cover the **highest-priority** topics (VQE, QEC, MS stack, chemistry pipeline). Projects 1, 2, 5, 7, and 8 form the essential core.

Prepared February 2026. Good luck, Edoardo!